Lecture 1
Transformations & the pinhole camera

Robot Control · University of Warsaw

Course logistics

Communication: Slack

Announcements and questions are on Slack: scan the code to join the workspace.

invitation link

Lab materials

mim-ml-teaching.github.io/public-rc-2026-27

Homework submissions

Moodle

Grading

Two parts — pass both

- pen & paper exam (theory): **30%**
- oral exam: **70%**

Passing a part = at least 50% of its points (the threshold may be adjusted down).

Homework

- 3 assignments, presented and defended during the oral exam
- you may submit during the semester to get feedback — only submissions made by the deadline are guaranteed feedback

Projects instead of homework

Some homework assignments can be replaced by an individually defined project (e.g. hardware deployment). The proposal must be approved by one of the lab teachers.

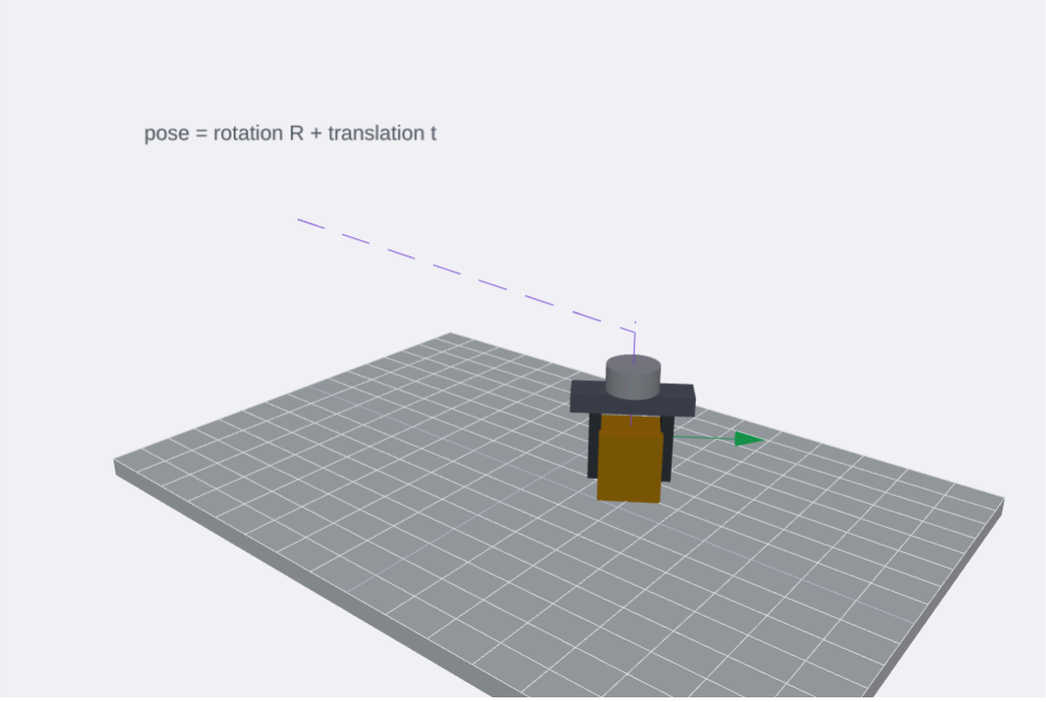
Lab bonus

For exceptional commitment, your lab teacher can add half a grade at the end of the semester (e.g. 4+ → 5). Ask your lab teacher how the bonus is judged.

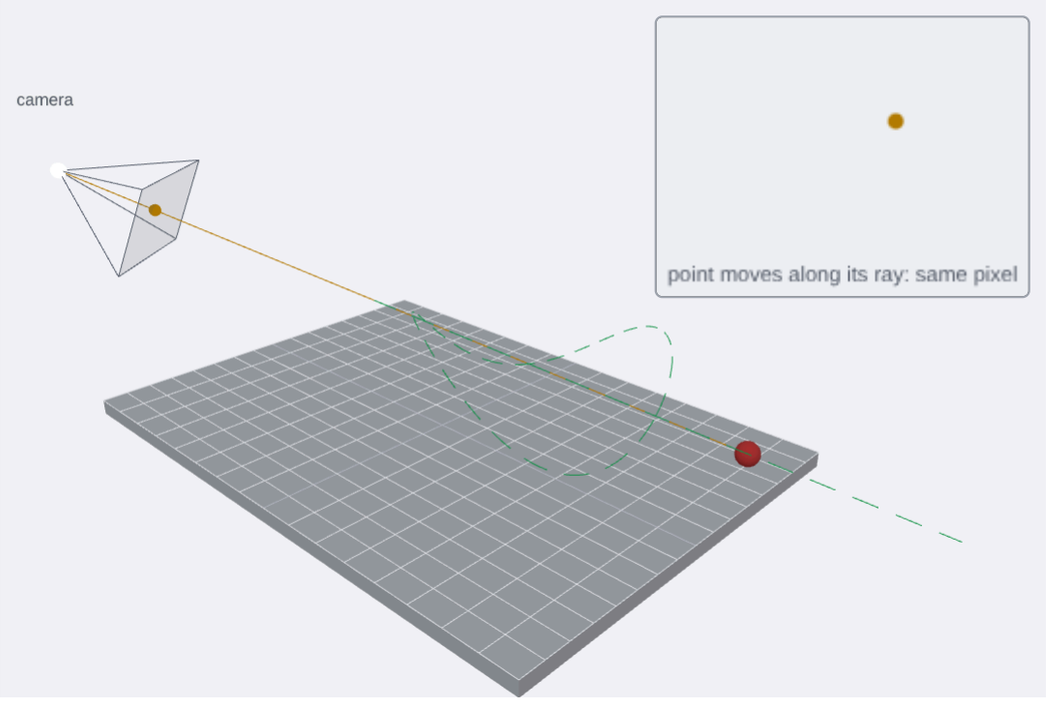
Note: $2 + 0.5 = 2.5 < 3$.

Today we mathematically model

Moving things around



Taking a picture



Today we mathematically model: the plan

Moving things around

- 2D image transformations as matrices
- homogeneous coordinates
- rigid motion in 3D: $SO(3)$, $SE(3)$
- how to represent a rotation

Taking a picture

- pinhole camera: 3D point → pixel
- pixel → ray in 3D
- vanishing points
- lens distortion

> Goal: be able to do these computations **by hand** (lab sessions & exam) and in code.

Further reading: First Principles of Computer Vision (S. Nayar) — Imaging → Image formation; Szeliski, Computer Vision: Algorithms and Applications, ch. 2.

Part 1

2D image transformations

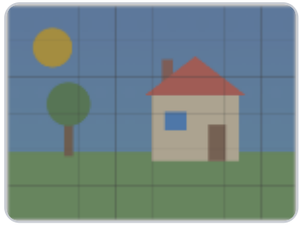
An image is a function

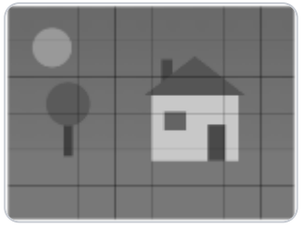
$f : \mathbb{R}^2 \to \mathbb{R}^3,\ (x, y) \mapsto (r, g, b)$. Pixel coordinates: origin top-left, x right, y **down**.

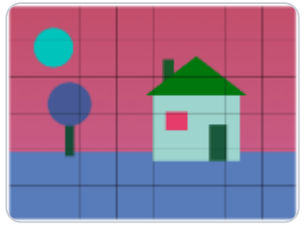
Photometric: change the values, $g(x) = h\big(f(x)\big)$, $h : \mathbb{R}^3 \to \mathbb{R}^3$ acts on colours, e.g. brightness $h(c) = 1.4\,c$


original

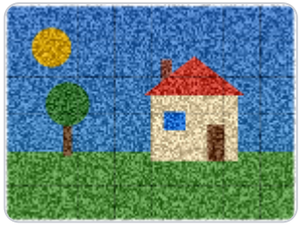

brightness +40%


contrast −50%


greyscale


hue shift

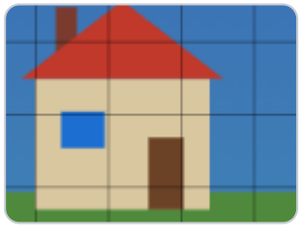

blur

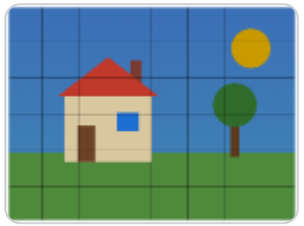

noise

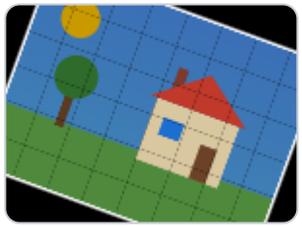
Geometric: transform the domain, $g(x) = f\big(T^{-1}(x)\big)$, $T : \mathbb{R}^2 \to \mathbb{R}^2$ moves pixel positions, e.g. horizontal flip

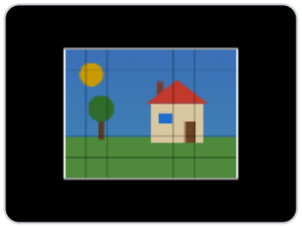
$T(x, y) = (W - x,\ y)$

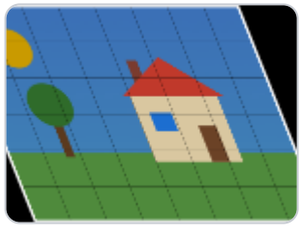

original

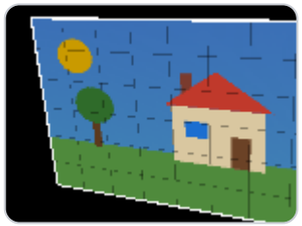

crop + resize


horizontal flip

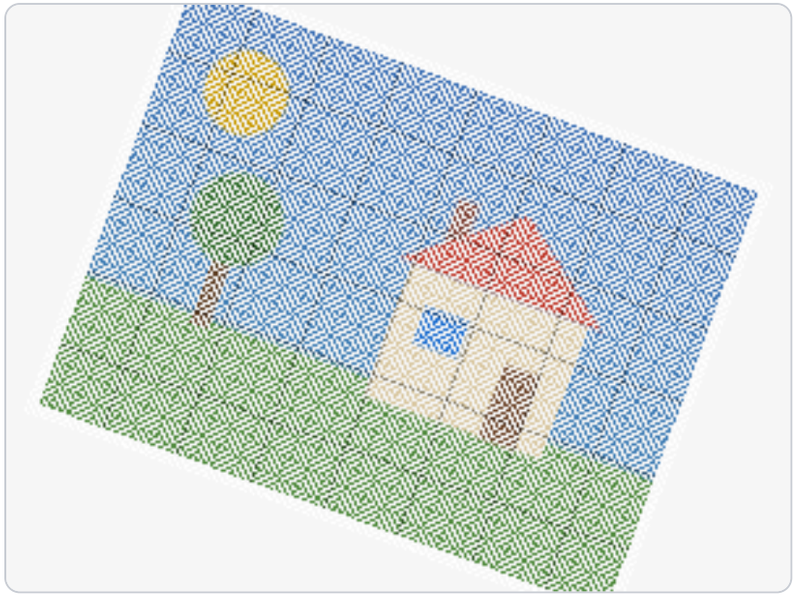

rotate 20°


scale ×0.6

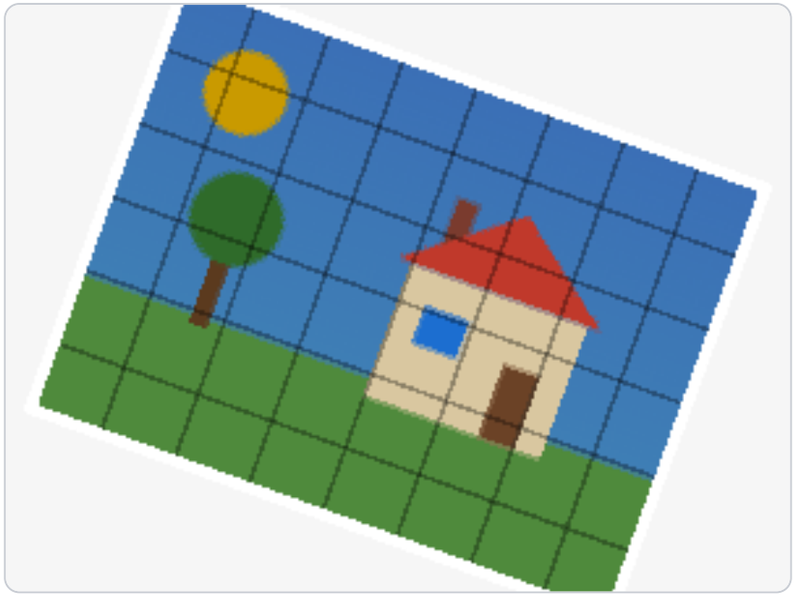

shear


perspective

Why T^{-1}?



forward: g(T(x)) = f(x)
loop over input pixels, push each one — holes



backward: g(x) = f(T⁻¹(x))
loop over output pixels, pull — no holes

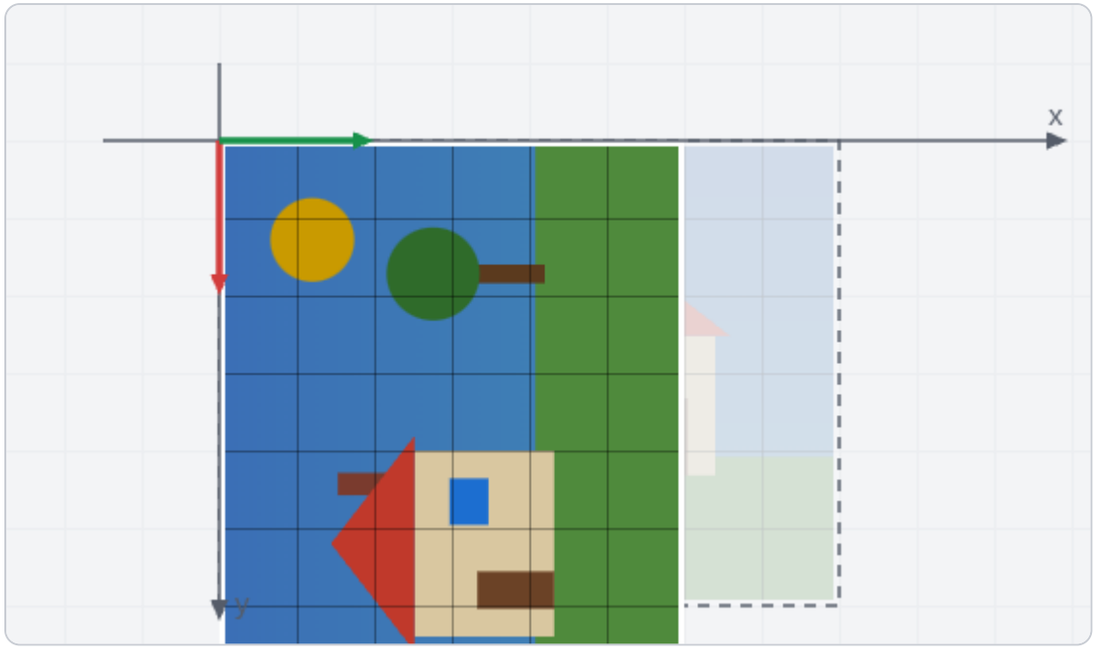
To colour output pixel x we ask: where did it come from?

$$g(x) = f\left(T^{-1}(x)\right)$$

This is **backward warping**; we come back to it for image stitching in lecture 3.

$T^{-1}(x)$ is rarely an integer pixel: we need **interpolation** (nearest neighbour here).

Linear maps: 2×2 matrices



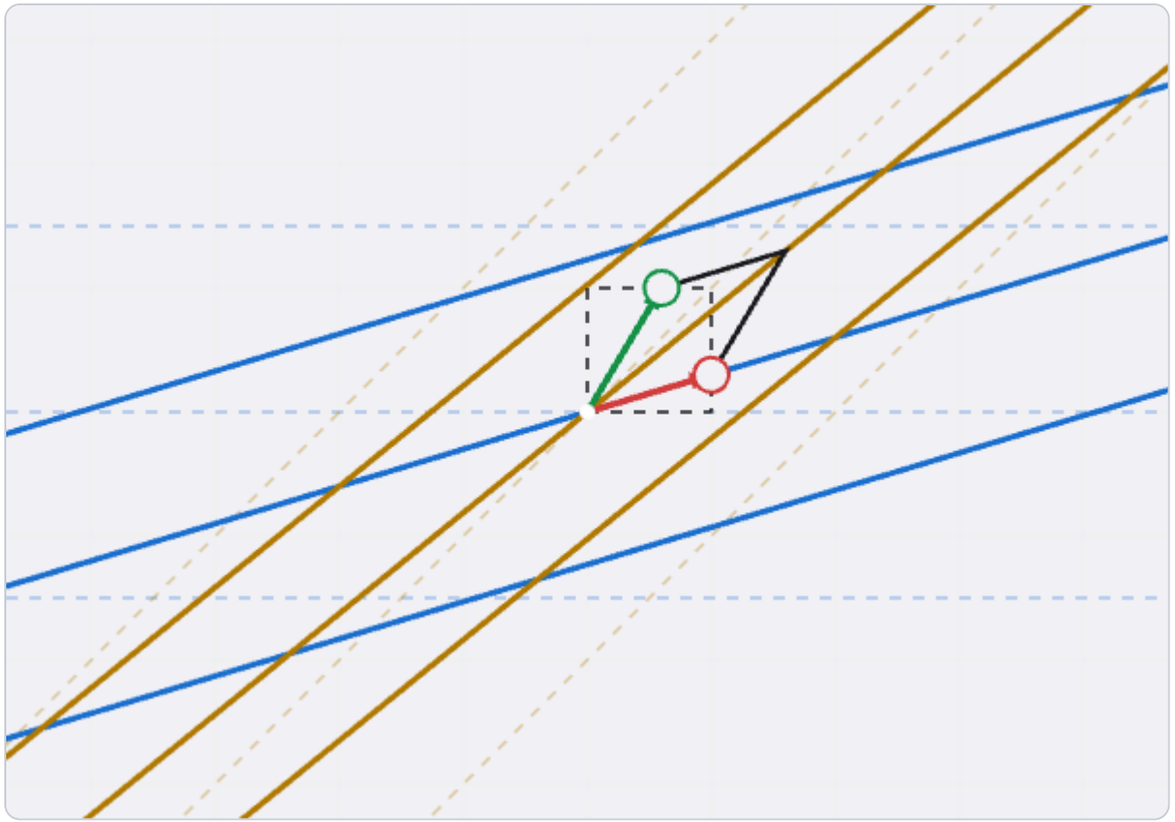
$$M = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

isometry with reflection — origin fixed (linear)

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

- the columns are the images of e_1 and e_2
- origin ↦ origin; lines ↦ lines, parallel lines stay parallel
- composition = matrix product: $T_{31} = T_{32}\, T_{21}$

Lines stay lines, parallel lines stay parallel



$$M = \begin{bmatrix} 1 & 0.6 \\ 0.3 & 1 \end{bmatrix}$$

linear: straight lines stay straight, parallel lines stay parallel, the origin stays put. Drag the red and green arrow tips (= columns of M).

dashed: before · solid: after

Why: $M(p + t\,d) = Mp + t\,(Md)$ — the line through p with direction d becomes the line through Mp with direction Md. Parallel lines share d, so their images share Md.

Translation is not linear

$$x_2 = x_1 + t_x, \qquad y_2 = y_1 + t_y$$

> **Question** Which 2×2 matrix does this?

None: every linear map sends $0 \mapsto 0$.

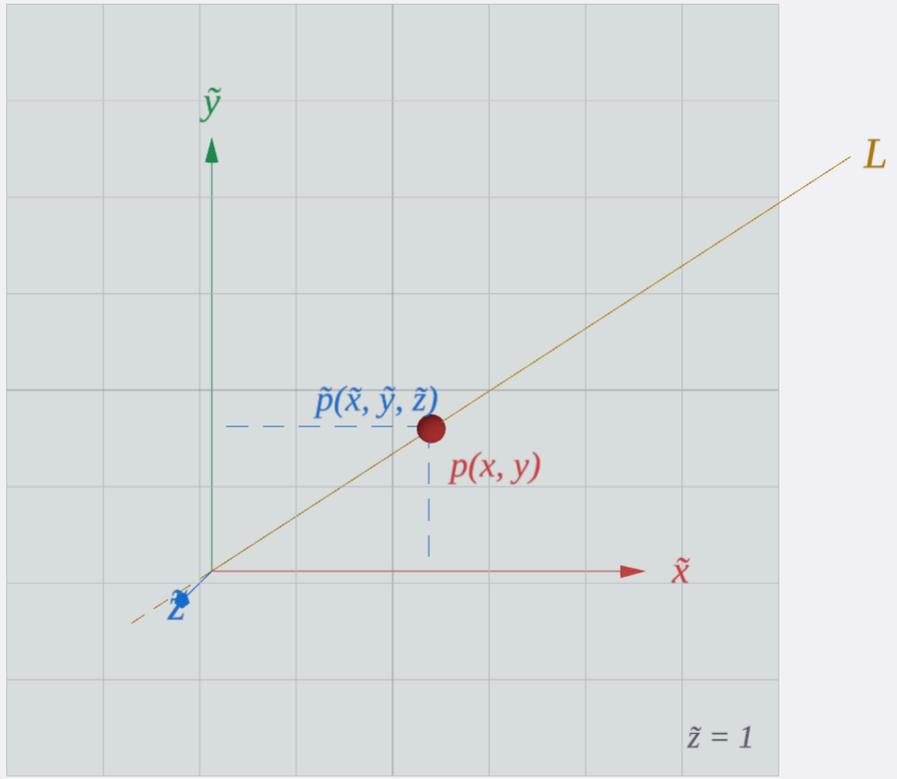
Trick: go one dimension up.

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

Advantage

One representation for everything: rotation, scaling, translation and projection all become matrix products, and a chain of them collapses into one matrix.

Homogeneous coordinates




```
p̃ = (1.2, 0.8, 1)
p = (x̃/z̃, ỹ/z̃) = (1.2, 0.8)
```

A 2D point $p = (x, y)$ is represented by any

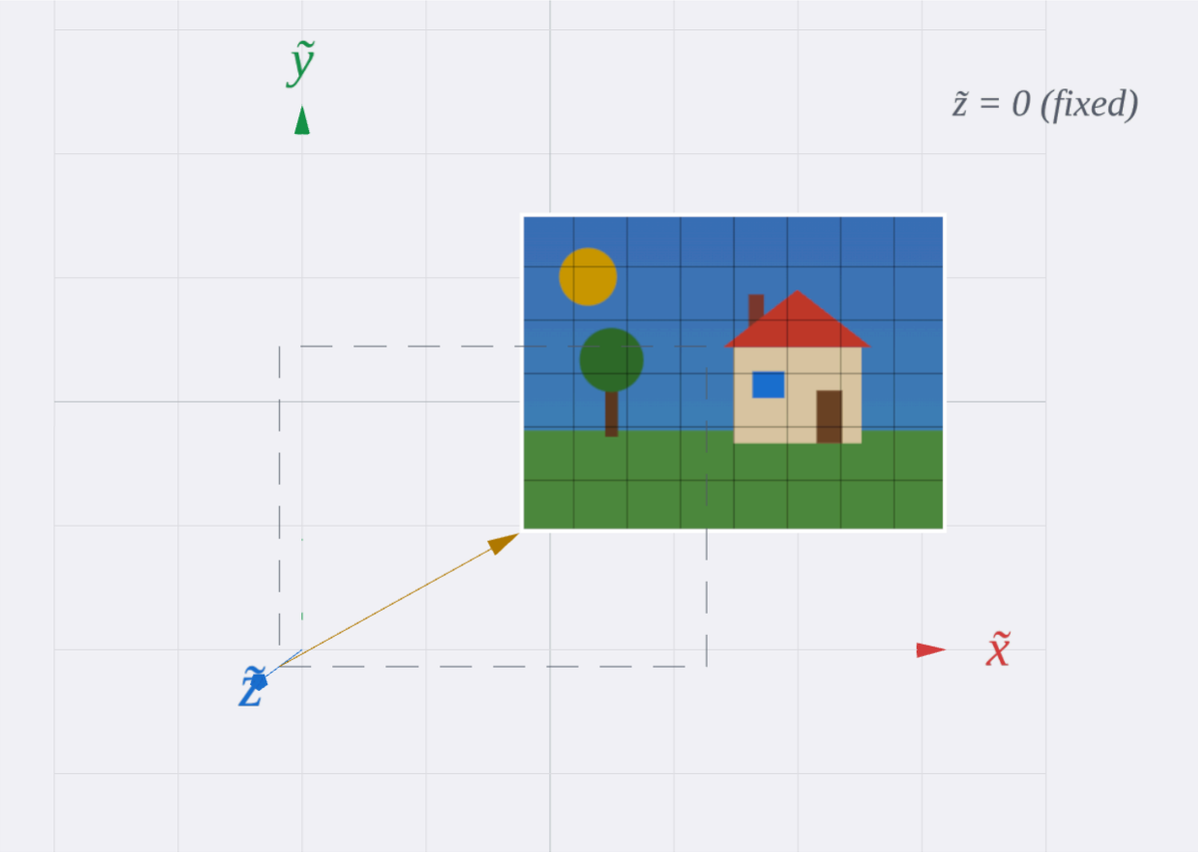
$$\tilde{p} = (\tilde{x}, \tilde{y}, \tilde{z}), \quad \tilde{z} \neq 0$$

with $x = \tilde{x}/\tilde{z},\ y = \tilde{y}/\tilde{z}$.

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \equiv \begin{bmatrix} \lambda x \\ \lambda y \\ \lambda \end{bmatrix}$$

A point = a ray through the origin.
Its intersection with the plane $\tilde{z} = 1$ is the ordinary point.

Translation in 2D is a linear map in 3D



$$\begin{bmatrix} 1 & 0 & 0.9 \\ 0 & 1 & 0.5 \\ 0 & 0 & 1 \end{bmatrix}$$

plane z̃ = 1 moves by (0.9, 0.5)

plane z̃ = 0 does not move

The 3×3 translation matrix is a **shear** of 3D space:

$$(\tilde{x}, \tilde{y}, \tilde{z}) \mapsto (\tilde{x} + t_x\tilde{z},\ \tilde{y} + t_y\tilde{z},\ \tilde{z})$$

- linear: origin fixed, lines stay lines
- plane $\tilde{z} = 0$ does not move
- plane $\tilde{z} = 1$ (our image) slides by (t_x, t_y)

Match the matrices

A: $\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$ scaling

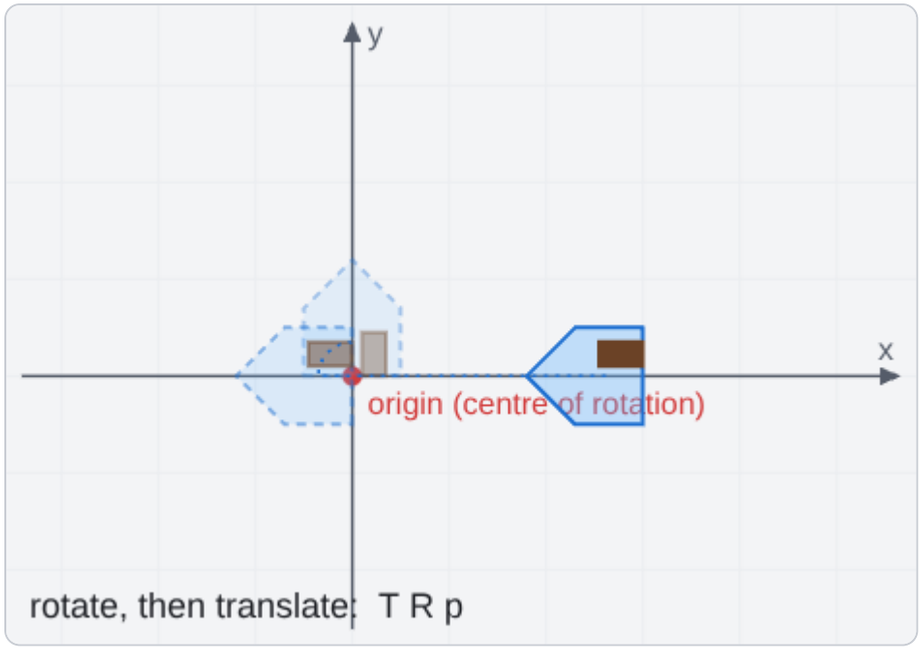
B: $\begin{bmatrix} 1 & m & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ skew (shear)

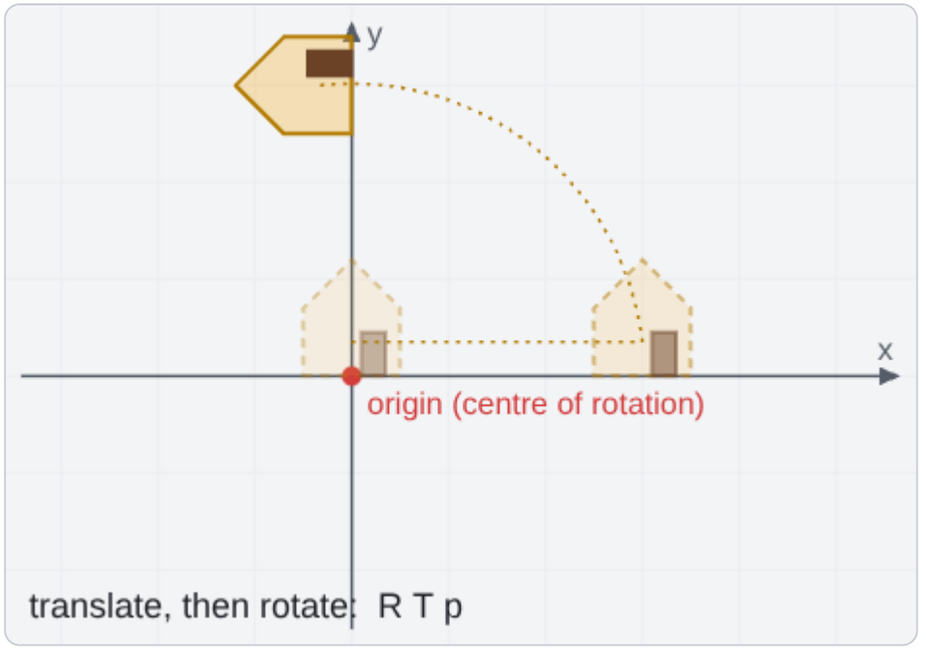
C: $\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$ translation

D: $\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$ rotation

Options: rotation · scaling · translation · skew.

Order matters





$$TR = \begin{bmatrix} R_{2\times2} & t \\ 0 & 1 \end{bmatrix} \quad \text{vs} \quad RT = \begin{bmatrix} R_{2\times2} & R_{2\times2}\,t \\ 0 & 1 \end{bmatrix}$$

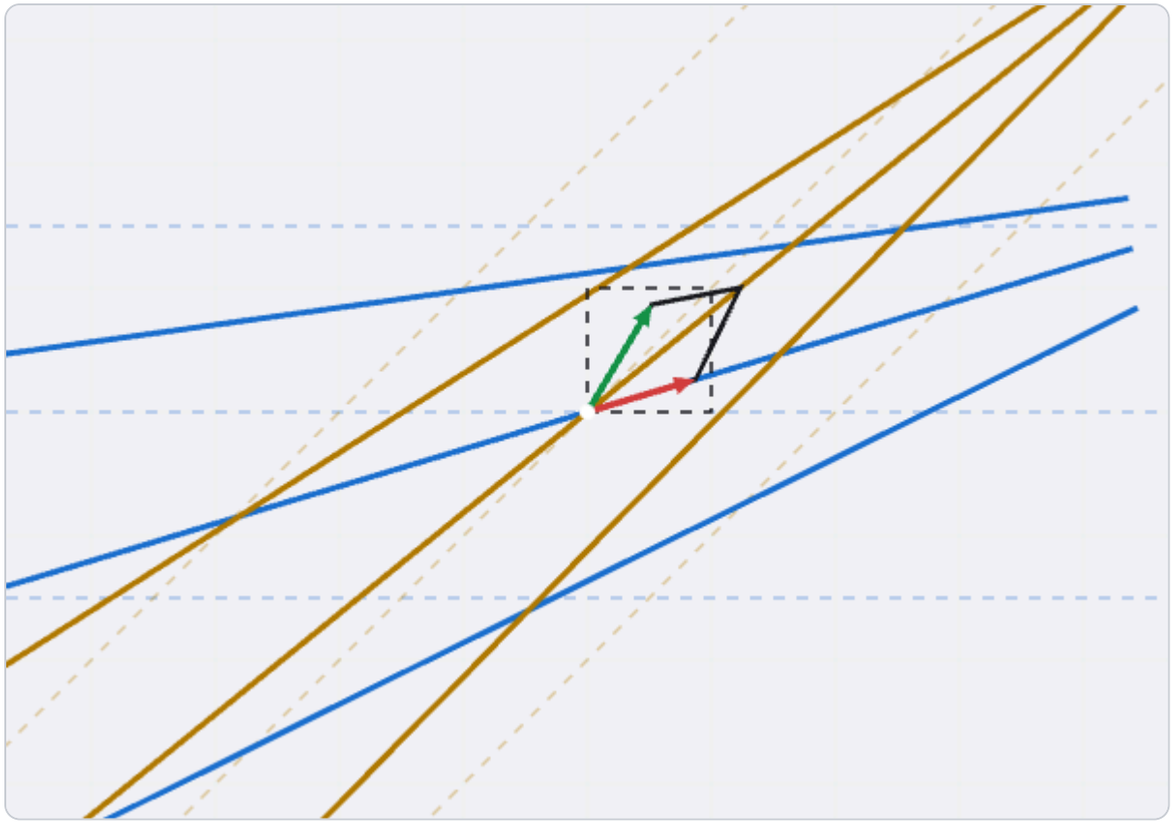
In general $TR \neq RT$. The matrix closest to p acts **first**; in RT the translation vector itself gets rotated.

A hierarchy of 2D transformations

Transformation	3×3 matrix representation	DoF	Preserves
translation	$\begin{bmatrix} I & t \\ 0^\top & 1 \end{bmatrix}$	2	orientation + everything below
rigid (Euclidean)	$\begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix}$	3	lengths
similarity	$\begin{bmatrix} sR & t \\ 0^\top & 1 \end{bmatrix}$	4	angles, ratios of lengths
affine	$\begin{bmatrix} A & t \\ 0^\top & 1 \end{bmatrix}$	6	parallelism, ratios of areas
projective (homography)	$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$	8	straight lines

Each row contains the ones above it. Homography: 9 entries but defined only up to scale ($H \equiv kH$), hence 8 DoF. We use it for image stitching in lecture 3.

Beyond affine: projective and non-linear maps OPTIONAL



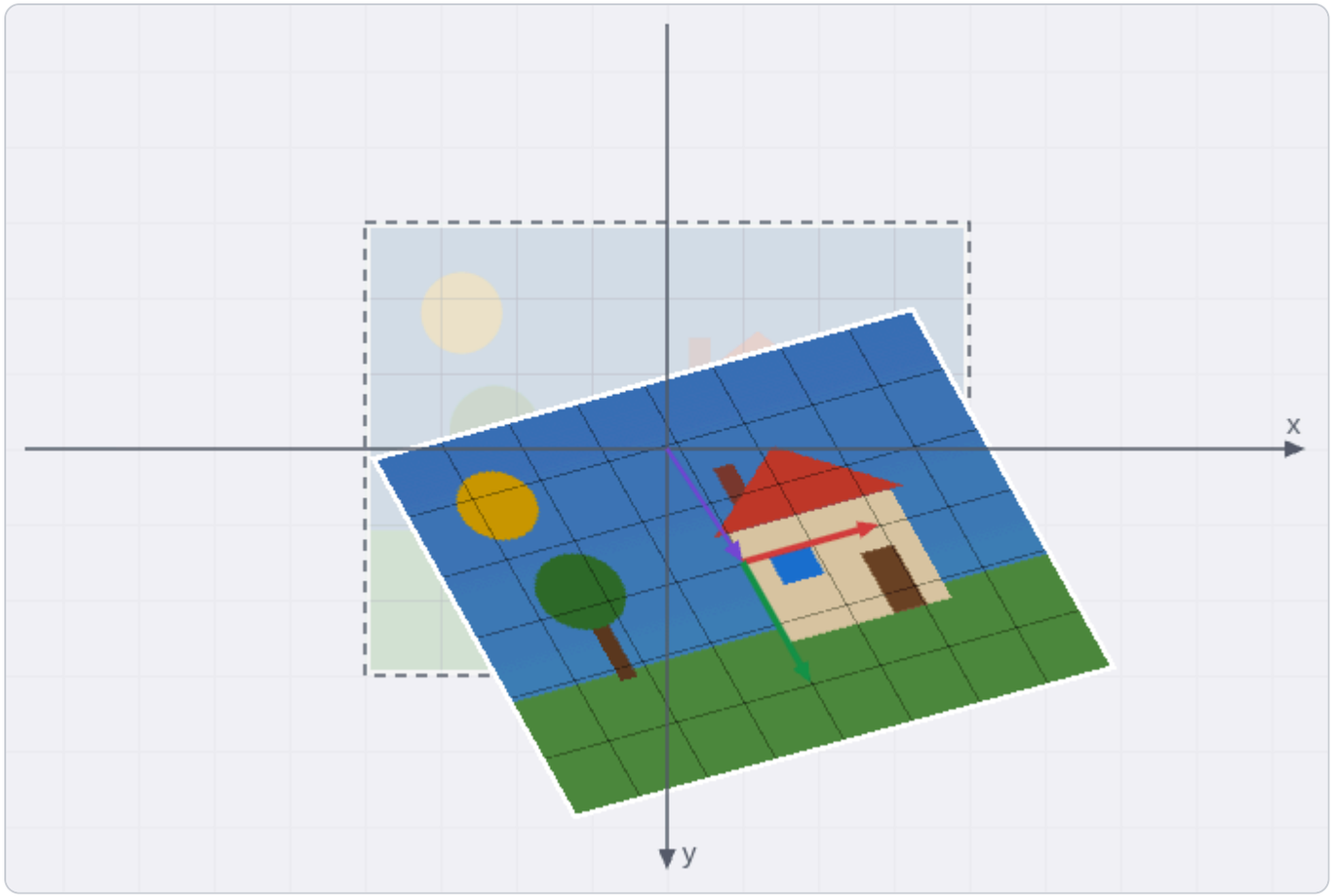
$$H = \begin{bmatrix} 1 & 0.6 & 0 \\ 0.3 & 1 & 0 \\ 0.14 & 0.15 & 1 \end{bmatrix}$$

projective: lines stay straight, but parallel lines now meet.

dashed: before · solid: after

Projective: lines stay lines, but parallel lines may meet. Non-linear: lines bend, and no matrix describes the map.

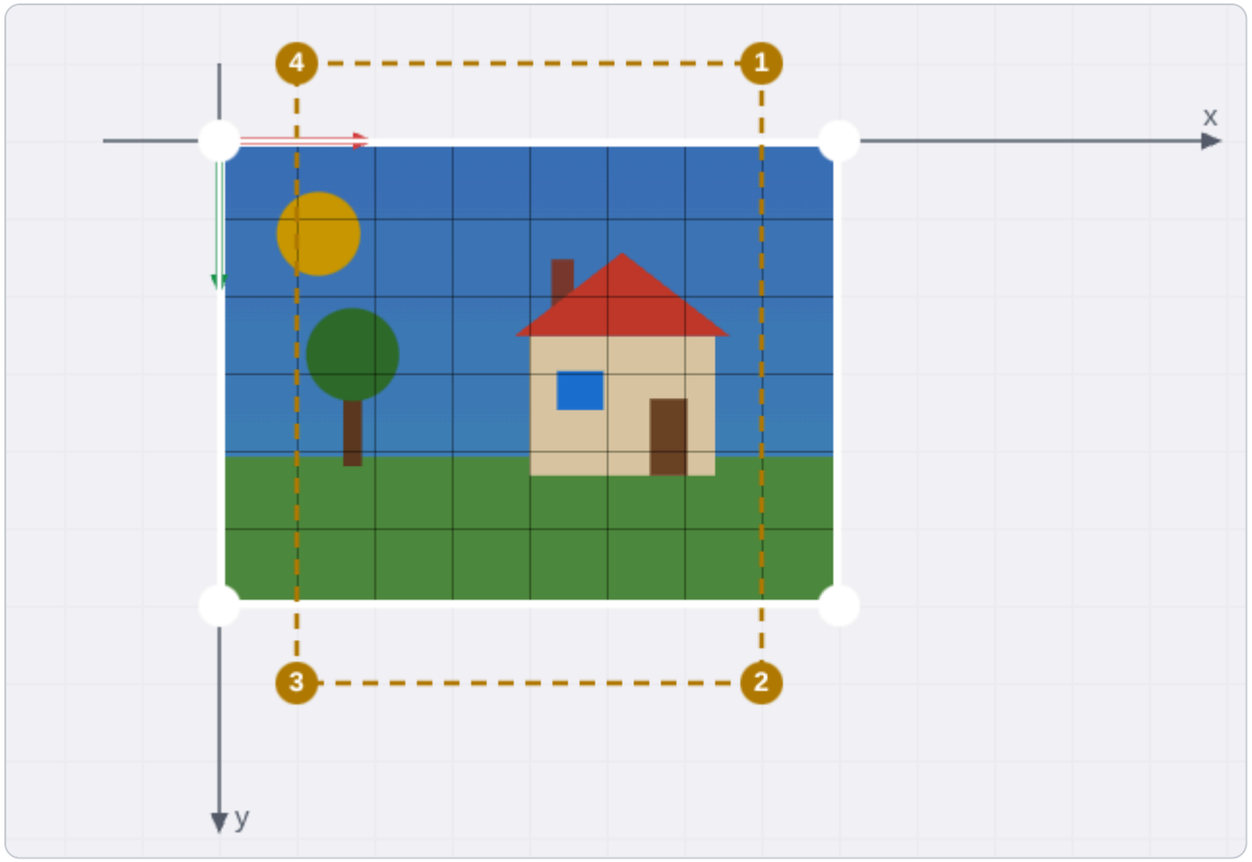
Playground



$$M = \begin{bmatrix} 0.9 & 0.45 & 40 \\ -0.25 & 0.8 & 60 \\ 0 & 0 & 1 \end{bmatrix}$$

affine

Exercise: reach the goal



Task 1 | Task 2 | Task 3

Rotate the image by 90° about its centre (160, 120).

$$M = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

✗ not yet

max corner error: 282.8 px

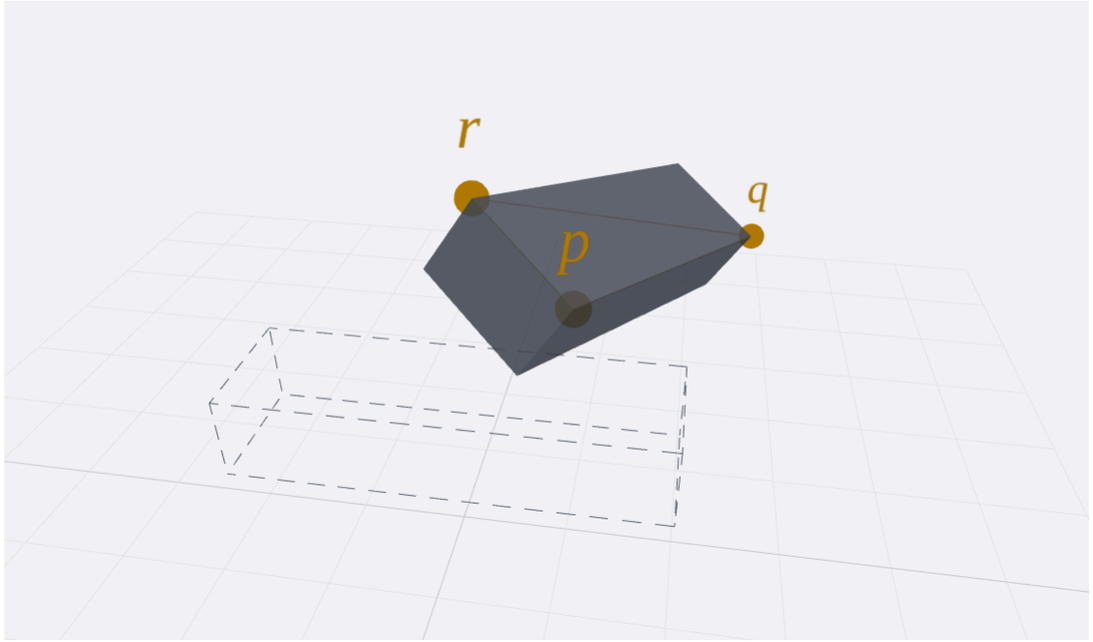
rigid (rotation + translation) — origin fixed (linear)

Part 2

Rigid body motion in 3D

Moving a rigid object

A robot moves a grasped object. The motion is a map $g : \mathbb{R}^3 \to \mathbb{R}^3$ applied to **all** points of the object.



distance	before	now	
|p − q|	2	2	✓
|q − r|	2.09	2.09	✓
|p − r|	0.6	0.6	✓

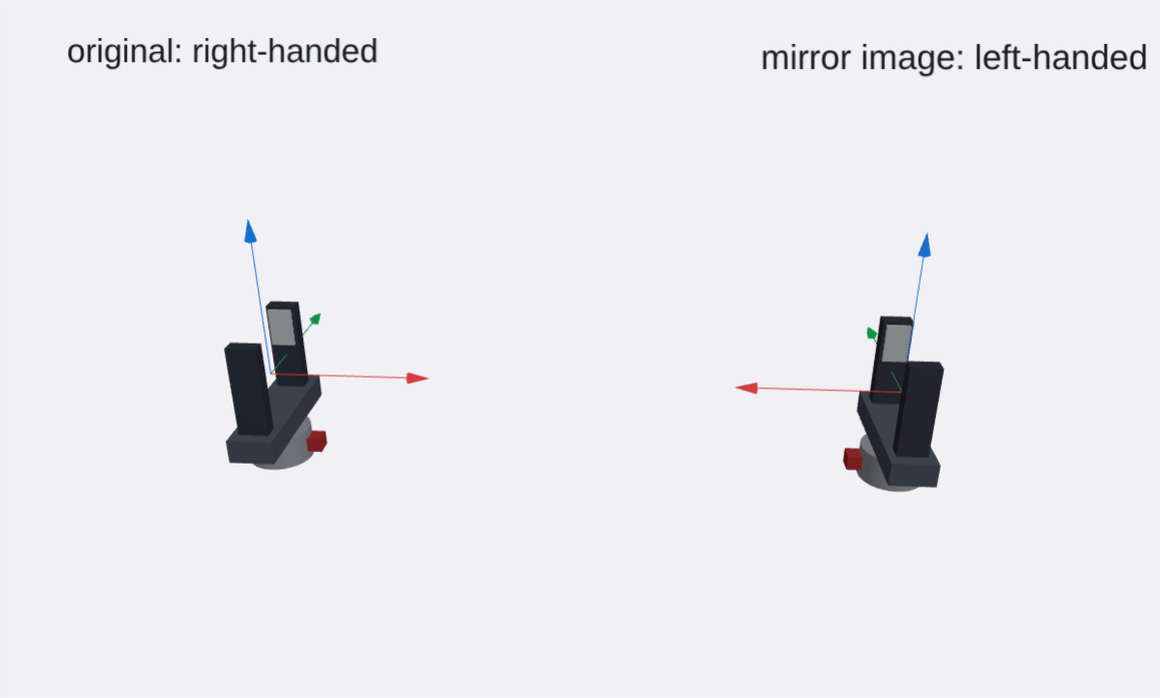
all distances preserved

dashed: starting position

Question Which mathematical property says "the object does not bend"? → distances are preserved, $\|g(p) - g(q)\| = \|p - q\|$ for all p, q: an **isometry**.

Question Is every isometry a motion we can perform with a real object?

A mirror image is also an isometry



det = −1 for the reflection, +1 for every rotation: no rotation can undo it.

- A reflection, e.g. $g(x) = \mathrm{diag}(-1, 1, 1)\, x$, preserves all distances — but **no rotation undoes it**: it turns a right-handed frame into a left-handed one.
- A real object can only be rotated and translated, so rigid motions must also preserve **orientation** (handedness). Next slide: this is exactly $\det A = +1$.

From "preserves distances" to $Rx + t$

1. **Mazur–Ulam**: every (surjective) isometry of $\mathbb{R}^n$ is affine: $g(x) = Ax + t$.
2. A preserves lengths, hence dot products (polarisation): $\langle Ax, Ay \rangle = \langle x, y \rangle \;\Rightarrow\; A^\top A = I, \quad A$ is **orthogonal**.
3. $\det(A^\top A) = \det(A)^2 = 1 \;\Rightarrow\; \det A = \pm 1$.
   - $\det A = +1$: rotation · $\det A = -1$: rotation composed with a reflection
4. Excluding reflections: $A \in SO(3) = \{R \in \mathbb{R}^{3\times 3} : R^\top R = I,\ \det R = 1\}$

> Rigid motion = rotation + translation: $\quad g(x) = Rx + t, \quad R \in SO(3)$

From "preserves distances" to $Rx + t$: why OPTIONAL

1. **Isometries are affine** (Mazur–Ulam): every surjective isometry of $\mathbb{R}^n$ is $g(x) = Ax + t$.

   Idea: the midpoint of p, q is the only point at distance $\frac{1}{2}\|p - q\|$ from both, so g maps midpoints to midpoints; with continuity this forces g to be affine. Here $t = g(0)$ is where the origin goes and $Ax = g(x) - g(0)$.

2. **A preserves lengths**: $\|Ax\| = \|g(x) - g(0)\| = \|x - 0\| = \|x\|$.

   Lengths determine dot products (polarisation): $\langle x, y\rangle = \frac{1}{2}\left(\|x + y\|^2 - \|x\|^2 - \|y\|^2\right)$. Hence $\langle Ax, Ay\rangle = \langle x, y\rangle$, i.e. $x^\top A^\top A\, y = x^\top y$ for all x, y, so $A^\top A = I$: A is **orthogonal** (its columns are orthonormal).

3. **Only two values of $\det A$**: $\det(A^\top A) = \det(A)^2 = \det I = 1 \;\Rightarrow\; \det A = \pm 1$.

   $\det A = +1$: rotation $\cdot$ $\det A = -1$: rotation composed with a reflection, like the mirror image two slides back.

4. **A real object cannot be mirrored**, so $\det A = +1$: $A \in SO(3) = \{R \in \mathbb{R}^{3\times 3} : R^\top R = I,\ \det R = 1\}$.

> Rigid motion = rotation + translation: $\quad g(x) = Rx + t, \quad R \in SO(3)$

What the columns of R mean

$$R = \begin{bmatrix} | & | & | \\ r_1 & r_2 & r_3 \\ | & | & | \end{bmatrix}, \qquad r_i = R\, e_i$$

- column i = where the i-th axis goes
- columns are orthonormal: $r_i^\top r_j = \delta_{ij}$
- right-handed: $r_1 \times r_2 = r_3$

Two readings of $p_W = R\, p_B$

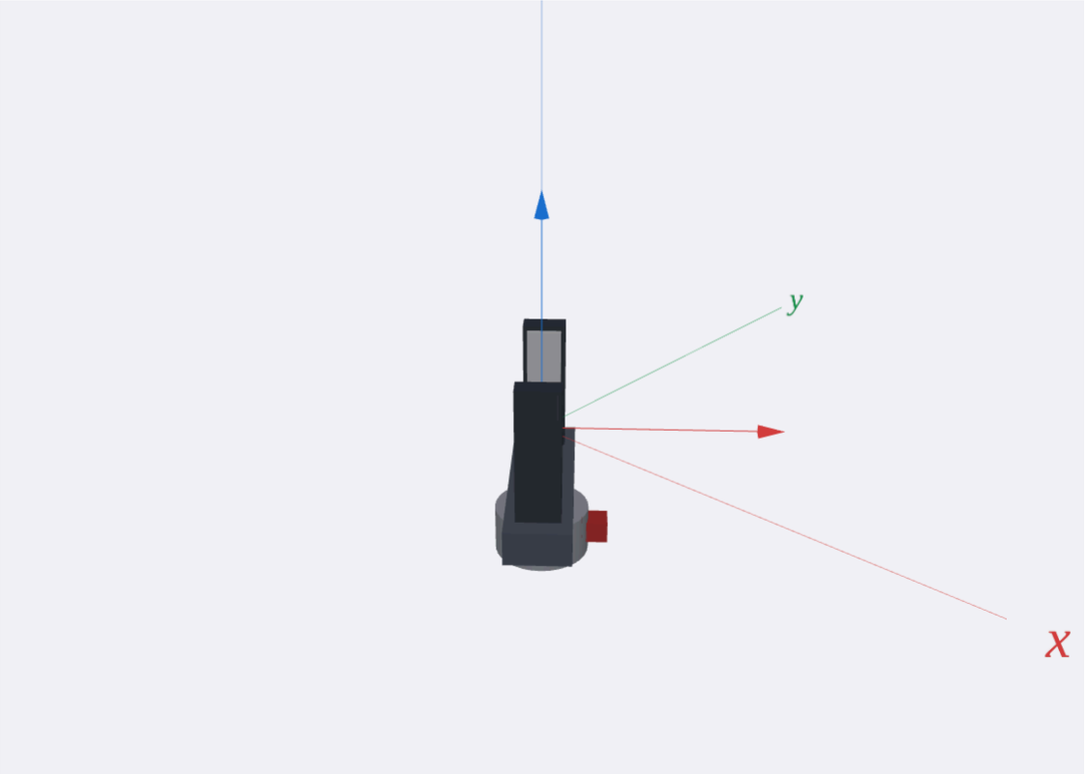
Active: rotate point p by R (same frame).

Passive: r_1, r_2, r_3 are the axes of a body frame B written in the world frame W; R converts coordinates from B to W.

Same matrix, two stories. Always write down which frame a vector lives in.

Properties of $SO(3)$: closed under products, $R^{-1} = R^\top \in SO(3)$, products do **not** commute.

Rotations about the coordinate axes OPTIONAL



$$R_z(40^\circ) = \begin{bmatrix} 0.77 & -0.64 & 0 \\ 0.64 & 0.77 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

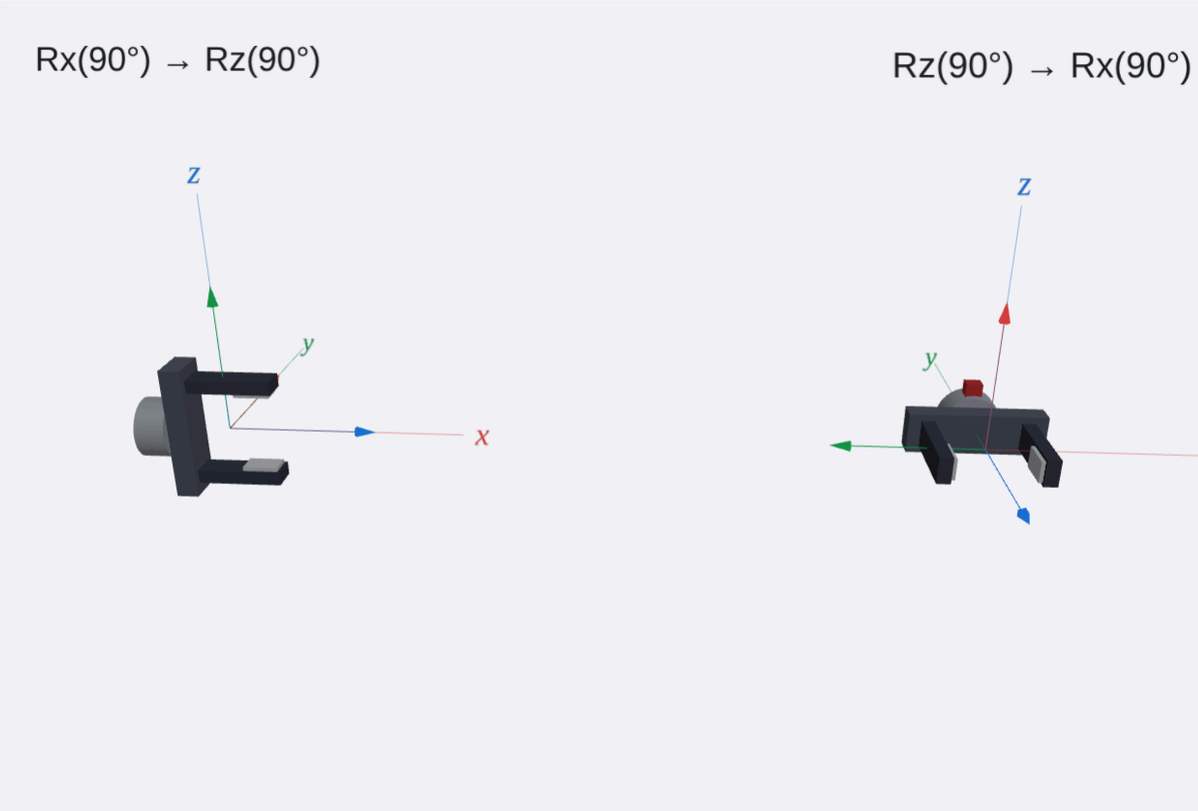
$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

R_y looks "flipped": the cyclic order is $x \to y \to z \to x$, so for R_y the plane is (z, x).

3D rotations do not commute OPTIONAL



left, R_x first: $R_z R_x = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$

right, R_z first: $R_x R_z = \begin{bmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{bmatrix}$

Homogeneous form: $SE(3)$

$$T = \begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix} \in SE(3), \qquad \begin{bmatrix} x' \\ 1 \end{bmatrix} = T \begin{bmatrix} x \\ 1 \end{bmatrix}$$

- composition: $T_2 T_1 = \begin{bmatrix} R_2 R_1 & R_2 t_1 + t_2 \\ 0^\top & 1 \end{bmatrix}$
- inverse: $T^{-1} = \begin{bmatrix} R^\top & -R^\top t \\ 0^\top & 1 \end{bmatrix}$ (not $-t$!)
- 6 degrees of freedom: 3 rotation + 3 translation

Notation used in this course

${}^A T_B$ maps coordinates in frame B to coordinates in frame A:

$$ {}^A p = {}^A T_B \, {}^B p$$

Chains cancel like fractions:

$$ {}^W T_C = {}^W T_R \, {}^R T_C$$

world ← robot ← camera: the camera pose of lecture 2.

Part 3

Representing rotations

How many numbers?

- R has 9 entries
- $R^\top R = I$ gives 6 independent constraints: 3 unit lengths + 3 orthogonality
- **3 degrees of freedom**

Every 3-number description has a singularity somewhere: no parametrisation of $SO(3)$ by 3 numbers is smooth and one-to-one everywhere. That is why we have several representations, each good for something.

representation	numbers	good for	watch out
rotation matrix	9	applying and composing	redundant; drifts off $SO(3)$ numerically
Euler angles	3	humans, joint angles, UIs	12 conventions, gimbal lock, bad interpolation
axis–angle / rotation vector	4 / 3	geometry, OpenCV `rvec`, small rotations	axis undefined at $\theta = 0$; composition not simple
unit quaternion	4	composing, interpolating, filtering	q and $-q$ are the same rotation

Euler angles

Three successive rotations about coordinate axes. The robotics / aerospace convention **ZYX** (yaw–pitch–roll):

$$R = R_z(\psi)\, R_y(\theta)\, R_x(\varphi)$$

- **intrinsic** reading: yaw about z, then pitch about the new y', then roll about x''
- **extrinsic** reading of the same product: roll about world x, then pitch about world y, then yaw about world z
- other conventions: ZYZ, XYZ, ... — 12 in total; always check which one a library uses

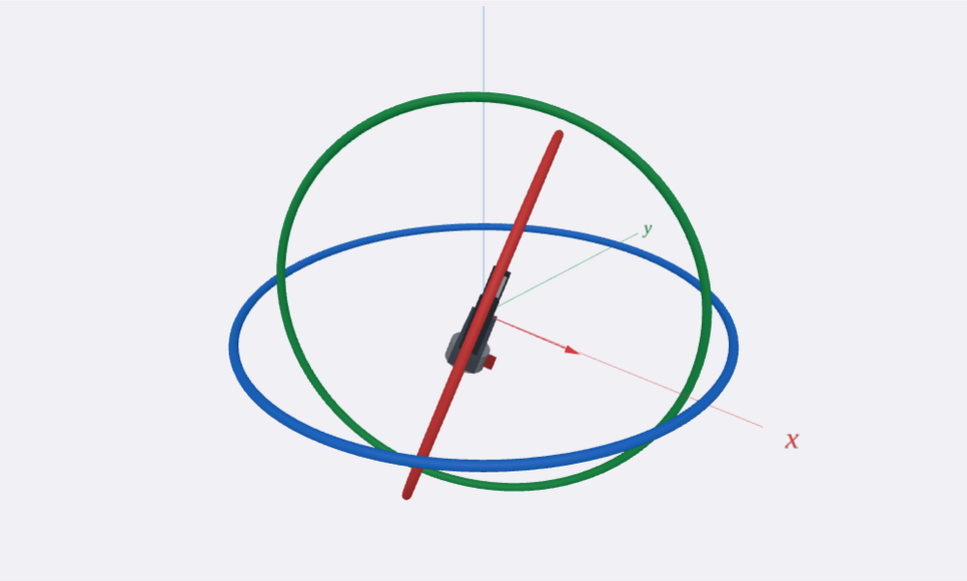
From R back to angles (ZYX)

$$\theta = -\arcsin r_{31}$$

$$\psi = \operatorname{atan2}(r_{21}, r_{11}), \quad \varphi = \operatorname{atan2}(r_{32}, r_{33})$$

Breaks down when $\cos\theta = 0$ …

Gimbal lock OPTIONAL



$$R = R_z(\psi)\, R_y(\theta)\, R_x(\varphi) = \begin{bmatrix} 0.81 & -0.5 & 0.3 \\ 0.47 & 0.87 & 0.17 \\ -0.34 & 0 & 0.94 \end{bmatrix}$$

What it is. At pitch $\theta = \pm 90^\circ$ the roll axis x'' lines up with the yaw axis z : the first and the last rotation turn about the **same axis** (two gimbal rings lie in one plane).

Not a property of the rotation, but of the **parametrisation**: every description of $SO(3)$ by 3 numbers has such singular points.

Mathematical consequences at the singularity:

- R depends only on $\varphi - \psi$: the map $(\psi, \theta, \varphi) \mapsto R$ loses rank (its Jacobian has rank 2) — one rotation direction cannot be reached by small changes of the angles
- the inverse is not unique: infinitely many (ψ, φ) give the same R; the atan2 formulas break down ($\cos\theta = 0$)
- near it, angle rates blow up: $\dot{\psi} = \dfrac{\omega_y \sin\varphi + \omega_z \cos\varphi}{\cos\theta}$ (ω: angular velocity in the body frame) — a small rotation needs huge angle changes

Gimbal lock: the algebra OPTIONAL

At $\theta = 90°$, $R_y(90°) = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}$ and

$$R_z(\psi)\, R_y(90°)\, R_x(\varphi) = \begin{bmatrix} 0 & \sin(\varphi - \psi) & \cos(\varphi - \psi) \\ 0 & \cos(\varphi - \psi) & -\sin(\varphi - \psi) \\ -1 & 0 & 0 \end{bmatrix}$$

- only $\varphi - \psi$ matters: one degree of freedom is lost
- near the singularity small changes of R need huge changes of ψ, φ — a problem for controllers and filters

> Euler angles are fine for displaying an orientation, not for computing with it.

Axis–angle

Euler's rotation theorem. Every rotation of $\mathbb{R}^3$ is a rotation by an angle θ about a fixed axis k through the origin ($\|k\| = 1$).

- axis + angle: 4 numbers with $\|k\| = 1$ → 3 DoF
- **rotation vector** $\omega = \theta k$: 3 numbers, $\theta = \|\omega\|$
- this is what OpenCV calls `rvec` (`cv2.Rodrigues`)

Why is there an axis?

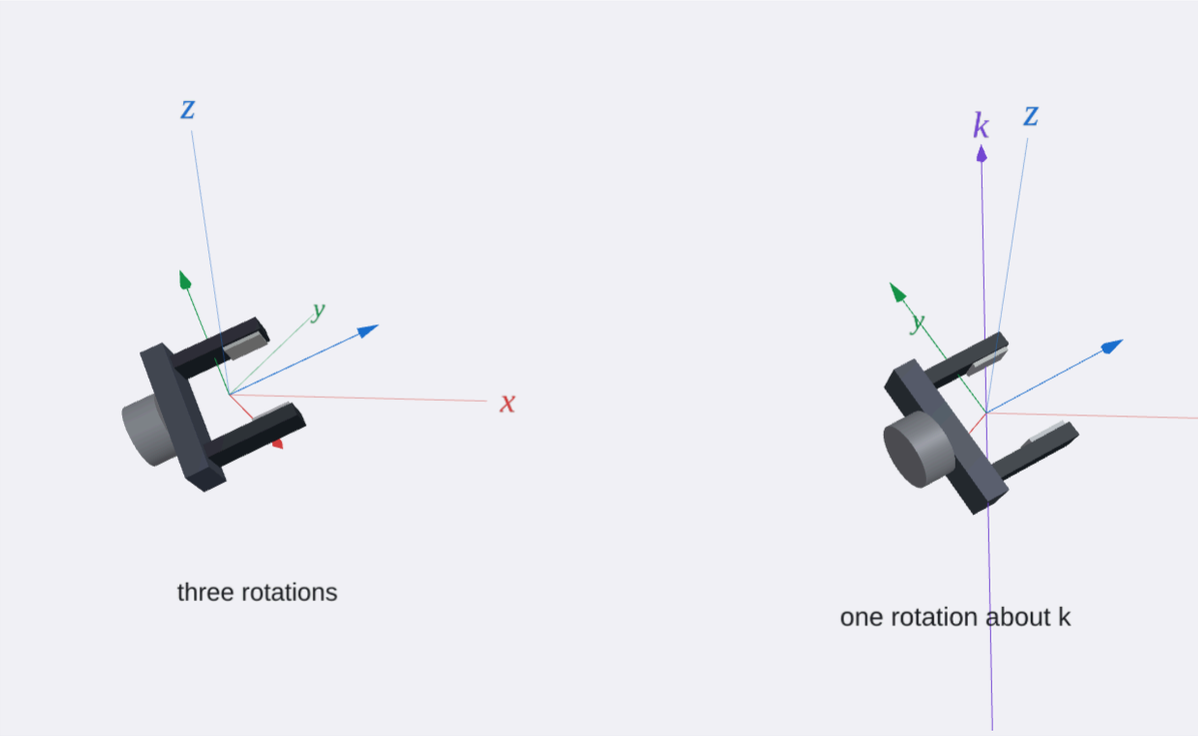
The axis is a vector that R does not move: $Rk = k$, i.e. 1 is an eigenvalue of R.

$$\begin{aligned}\det(R - I) &= \det(R^\top)\det(R - I) = \det(I - R^\top) \\ &= \det\left((I - R)^\top\right) = \det(I - R) \\ &= (-1)^3 \det(R - I)\end{aligned}$$

hence $\det(R - I) = 0$.

Uses $\det R = 1$ and $n = 3$ odd.

Euler's theorem, visually



left: $R_x(60°)$, then $R_y(45°)$, then $R_z(90°)$

$$R = R_z R_y R_x = \begin{bmatrix} 0 & -0.5 & 0.87 \\ 0.71 & 0.61 & 0.35 \\ -0.71 & 0.61 & 0.35 \end{bmatrix}$$

right: $\theta = 91°$ about $k = (0.13, 0.79, 0.6)$

Three rotations about the coordinate axes, one after another (left), end in exactly the same pose as **one** rotation by θ about a single axis k (right). This holds for any product of rotations.

From a matrix to axis–angle OPTIONAL

Angle from the trace (the trace does not depend on the basis):

$$\operatorname{tr} R = 1 + 2\cos\theta$$

Axis from the antisymmetric part:

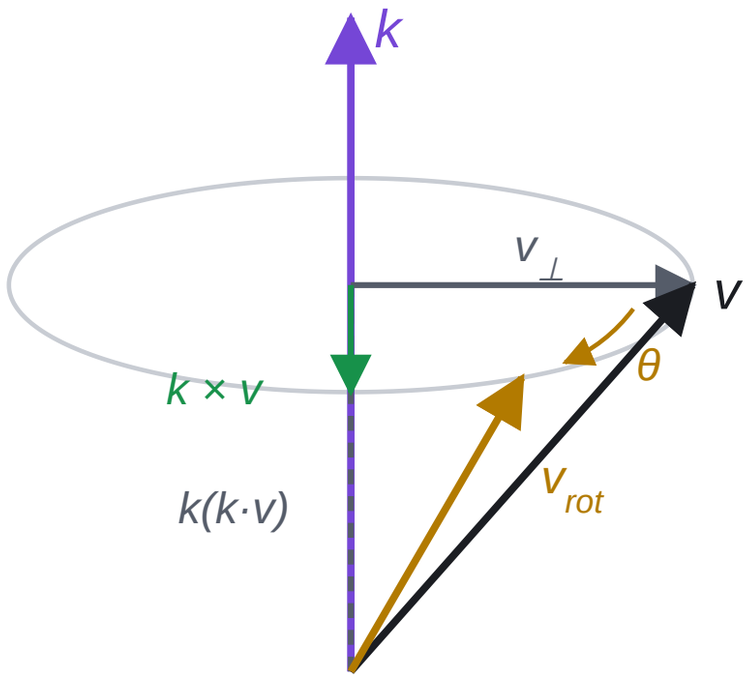
$$\frac{R - R^\top}{2} = \sin\theta\,[k]_\times$$

where $[k]_\times = \begin{bmatrix} 0 & -k_3 & k_2 \\ k_3 & 0 & -k_1 \\ -k_2 & k_1 & 0 \end{bmatrix}$, $[k]_\times v = k \times v$.

Special cases

- $\theta = 0$: any axis works
- $\theta = \pi$: $\sin\theta = 0$; read k from $\frac{1}{2}(R + I) = kk^\top$
- (k, θ) and $(-k, -\theta)$ are the same rotation

From axis–angle to a matrix: Rodrigues' formula OPTIONAL



Split v into parts parallel and perpendicular to k. The parallel part stays, the perpendicular part rotates in the plane spanned by $v_\perp$ and $k \times v$:

$$v_{\text{rot}} = v\cos\theta + (k \times v)\sin\theta + k\,(k \cdot v)(1 - \cos\theta)$$

In matrix form:

$$R = I + \sin\theta\,[k]_\times + (1 - \cos\theta)\,[k]_\times^2$$

Exponential coordinates OPTIONAL

Spin with constant angular velocity ω for one second: every point obeys $\dot{p} = \omega \times p = [\omega]_\times p$. The solution:

$$p(1) = e^{[\omega]_\times} p(0), \qquad R = e^{[\omega]_\times} = \sum_{n \geq 0} \frac{[\omega]_\times^n}{n!}$$

With $\omega = \theta k$ and $[k]_\times^3 = -[k]_\times$ the series collapses to Rodrigues' formula.

Why it matters

- the rotation vector is the exponential coordinate
- $\omega \mapsto e^{[\omega]_\times}$ gives a local, unconstrained parametrisation around any R_0: $R = R_0\, e^{[\delta]_\times}$ — used by optimisers (calibration, SLAM) and Kalman filters later in the course
- inverse map: $\log R$ = axis–angle extraction from the previous slides

Unit quaternions OPTIONAL

$$q = \left(\cos\tfrac{\theta}{2},\ \sin\tfrac{\theta}{2}\,k\right) = (w, x, y, z), \quad \|q\| = 1$$

- rotate: $(0, v') = q\,(0, v)\,\bar{q}$
- compose: quaternion product $q_2 q_1$ (16 multiplications vs 27 for matrices)
- **double cover**: q and $-q$ give the same rotation
- no singularities, smooth interpolation (slerp)

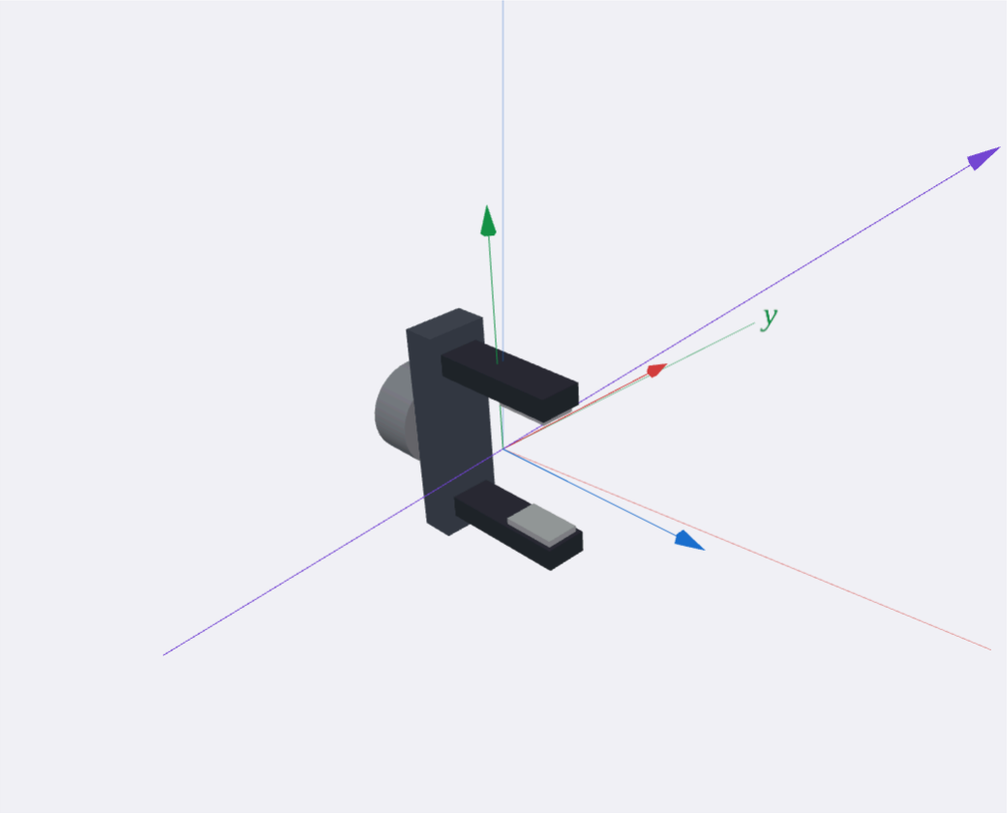
Quaternion product

For $q_i = (w_i, u_i)$ with scalar w_i and vector u_i:

$$q_1 q_2 = \left(w_1 w_2 - u_1 \cdot u_2,\ \ w_1 u_2 + w_2 u_1 + u_1 \times u_2\right)$$

Half angles: rotating by 2π gives $q = -1$, which is still the identity rotation.

One rotation, many descriptions



matrix: $\begin{bmatrix} 0.09 & 0 & 1 \\ 0.99 & -0.08 & -0.09 \\ 0.08 & 1 & -0.01 \end{bmatrix}$

axis–angle: $k = (0.63, 0.53, 0.57),\ \theta = 120^\circ$

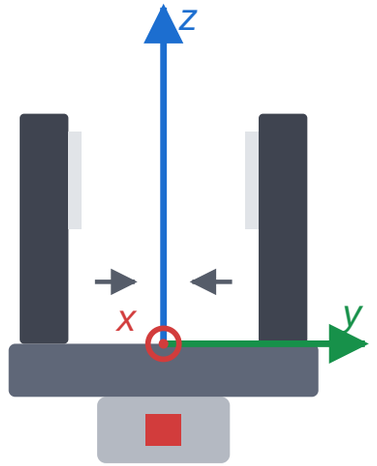
rotation vector: $\omega = \theta k = (1.31, 1.1, 1.2)$

quaternion (w, x, y, z): $(0.5, 0.54, 0.46, 0.5)$

Euler ZYX (ψ, θ, φ): $(85^\circ, -5^\circ, 90^\circ)$

z: approach direction

y: closing direction



x = y × z (out of the page)

Parallel-jaw gripper and its tool frame (the usual convention for robot end effectors).

Conventions in software

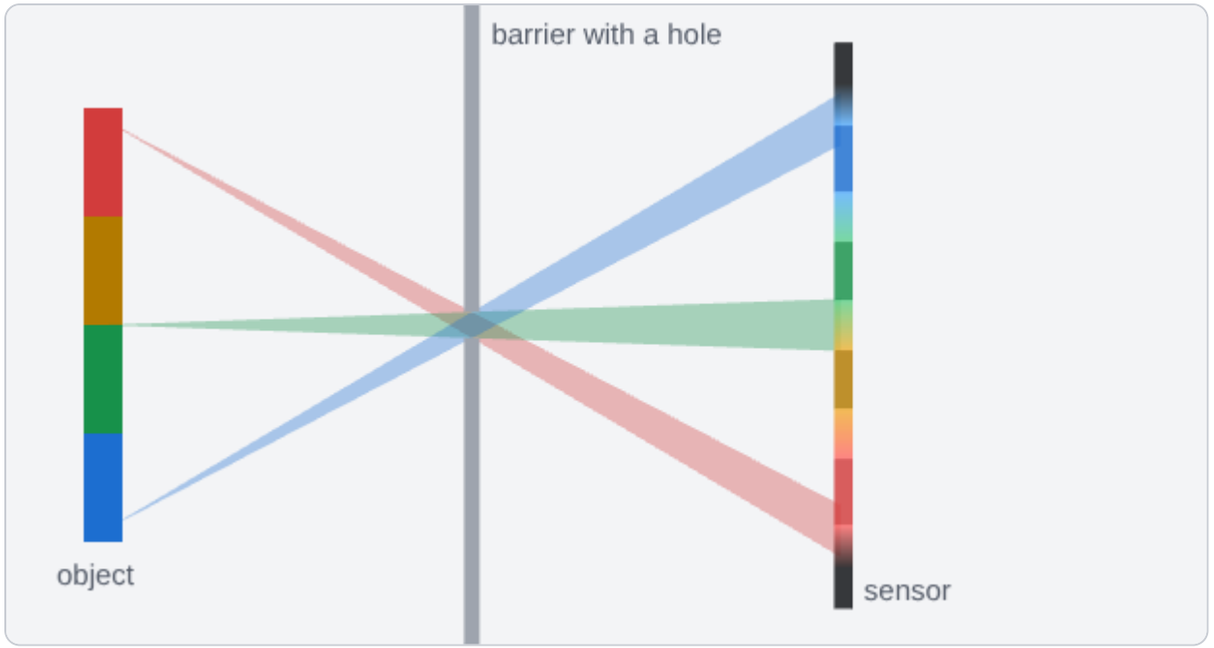
library	quaternion order	notes
ROS `geometry_msgs/Quaternion`	(x, y, z, w)	fields named explicitly
SciPy `Rotation.as_quat()`	(x, y, z, w)	scalar-last by default
Eigen `Quaterniond(w, x, y, z)`	constructor (w, x, y, z)	but `coeffs()` returns (x, y, z, w)
MuJoCo	(w, x, y, z)	
PyBullet	(x, y, z, w)	
OpenCV	—	rotation vector `rvec` + `cv2.Rodrigues`

> Before trusting a pose: check the order, the Euler convention, and whether a transform maps world→body or body→world.

Part 4

The pinhole camera

Why a pinhole?



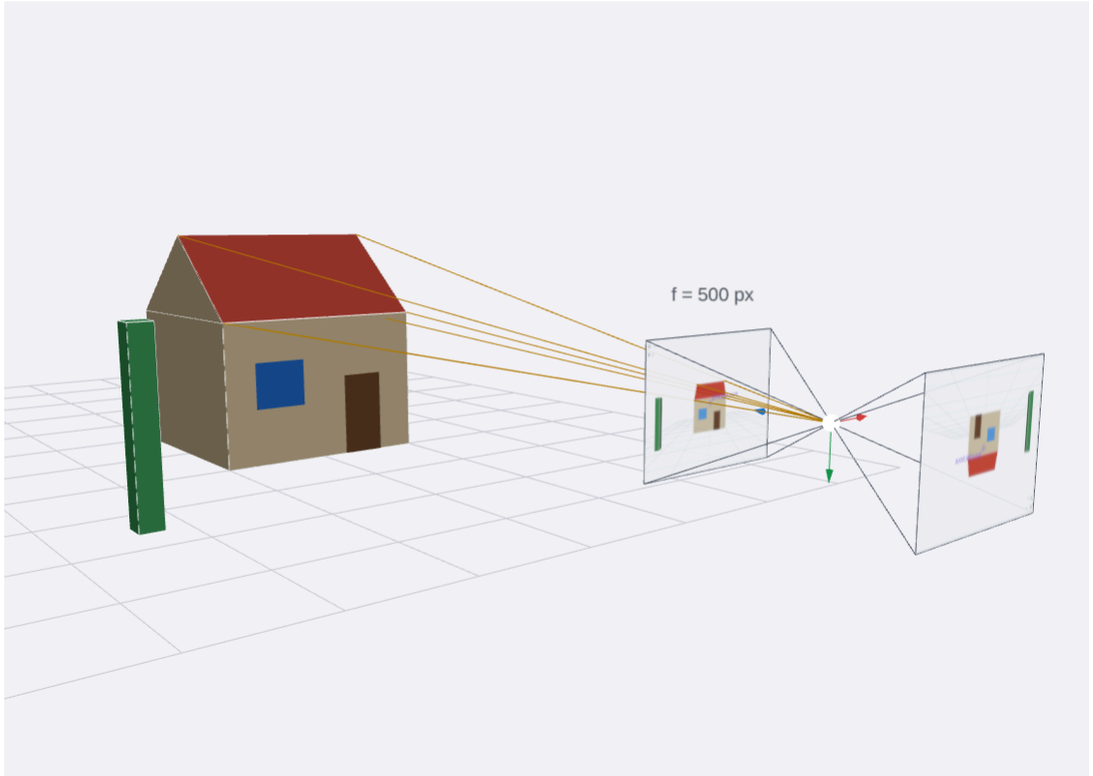
what the sensor records

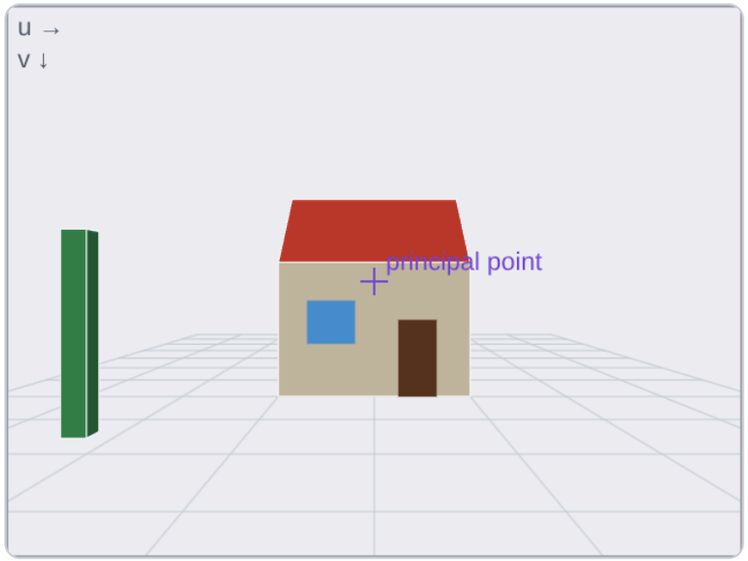


small hole: one ray per sensor point — a sharp image, **upside down**

- A bare sensor receives light from every point of the scene at every pixel: no image.
- Put a **barrier with a small hole** (the pinhole) in front of it: each sensor point now receives light along a single ray — a sharp image, **upside down**.
- Hole size trades **sharpness** for **brightness**; real cameras replace the hole with a lens (lecture 2 material).

The pinhole camera in 3D



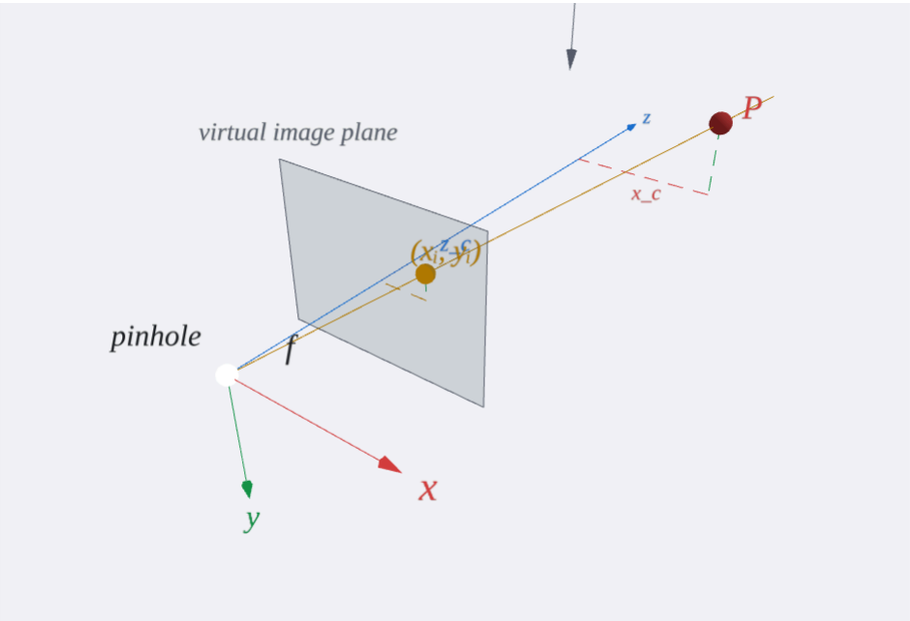


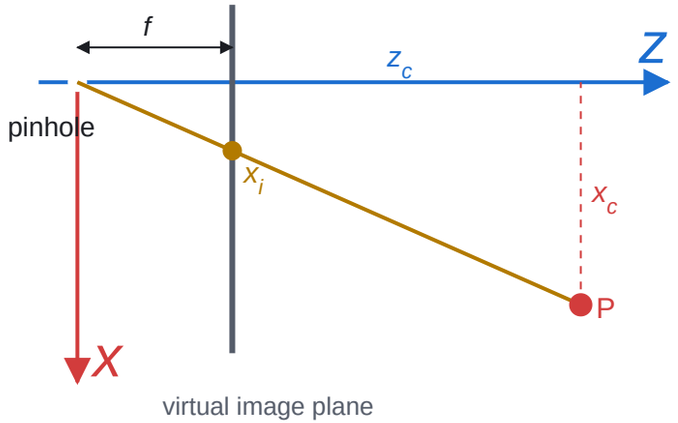
$$K = \begin{bmatrix} 500 & 0 & 320 \\ 0 & 500 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

The same in 3D: one ray through the pinhole per pixel, an inverted image on the physical image plane behind it. We work with the equivalent **virtual image plane** in front of the pinhole: same geometry, no flip.

Perspective projection

Camera frame: origin at the pinhole, z along the optical axis, x right, y down. A point $P = (x_c, y_c, z_c)$ and its image on the virtual plane at distance f:





top view: looking along y (from above)

Similar triangles: $\frac{x_i}{f} = \frac{x_c}{z_c}, \quad \frac{y_i}{f} = \frac{y_c}{z_c}$ (f: effective focal length; division by z_c: far things look small)

From millimetres to pixels

- the sensor has m_x, m_y pixels per mm
- pixel $(0, 0)$ is the top-left corner; the optical axis hits the sensor at the **principal point** (o_x, o_y)

$$u = \underbrace{m_x f}_{f_x} \frac{x_c}{z_c} + o_x, \qquad v = \underbrace{m_y f}_{f_y} \frac{y_c}{z_c} + o_y$$

Intrinsic parameters (f_x, f_y, o_x, o_y): the camera's internal geometry, in pixels.

Typical values

- $f_x \approx f_y$ (square pixels)
- $(o_x, o_y) \approx$ image centre, but not exactly
- field of view: $\mathrm{FOV}_x = 2 \arctan \frac{W}{2 f_x}$
- sometimes a 5th parameter, skew s (next lecture)

Projection as a matrix

The equations are non-linear (division by z_c), but linear in homogeneous coordinates:

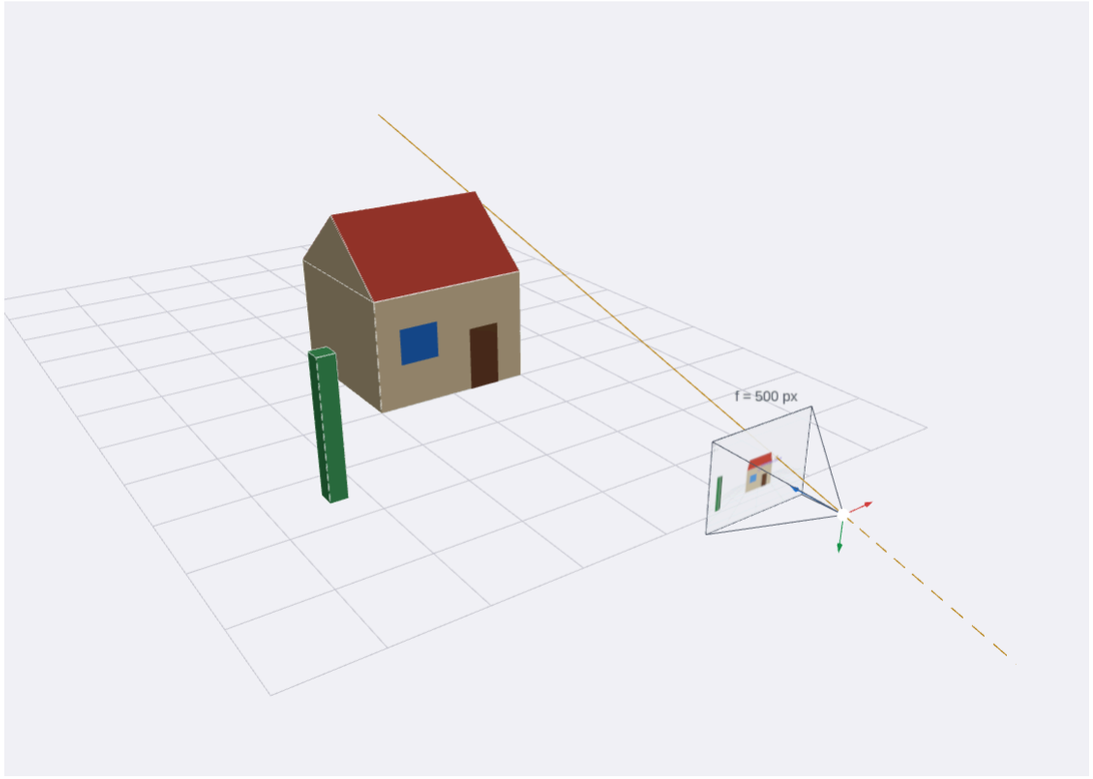
$$\begin{bmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{bmatrix} = \begin{bmatrix} z_c u \\ z_c v \\ z_c \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & o_x \\ 0 & f_y & o_y \\ 0 & 0 & 1 \end{bmatrix}}_{K} \begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix}, \qquad (u, v) = \left(\frac{\tilde{u}}{\tilde{w}}, \frac{\tilde{v}}{\tilde{w}} \right)$$

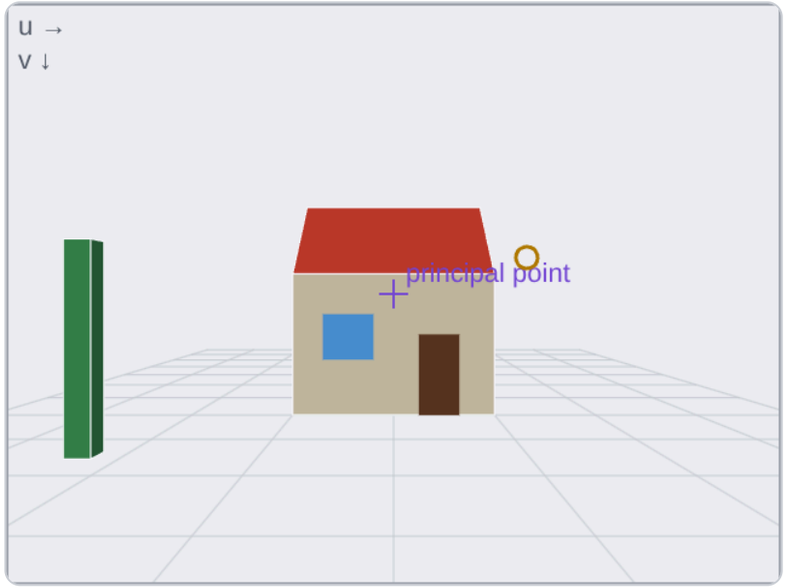
Both equalities are exact: Kx_c is the particular homogeneous vector $z_c\,(u, v, 1)$. As points, $(u, v, 1) \simeq Kx_c$ — equal up to scale.

- K: **calibration matrix**, upper triangular
- with a homogeneous 3D point: $\tilde{u} = [K \mid 0]\,\tilde{x}_c$

$[K \mid 0]$ is 3×4; next lecture we multiply it by the 4×4 camera pose to get the full projection matrix.

A pinhole camera to play with





$$K = \begin{bmatrix} 500 & 0 & 320 \\ 0 & 500 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

pixel $(430, 210)$ → ray $z\,(0.22,\ -0.06,\ 1),\ z > 0$

Worked example

$f_x = f_y = 800$, $(o_x, o_y) = (320, 240)$, point $x_c = (0.5, -0.25, 4)$ m.

$$u = 800 \cdot \frac{0.5}{4} + 320 = 420$$

$$v = 800 \cdot \frac{-0.25}{4} + 240 = 190$$

Back from pixel $(420, 190)$:

$$K^{-1} \begin{bmatrix} 420 \\ 190 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.125 \\ -0.0625 \\ 1 \end{bmatrix}$$

We get only the **direction**; any $z_c > 0$ fits.

With $z_c = 4$ we recover the point.

Pixel → ray

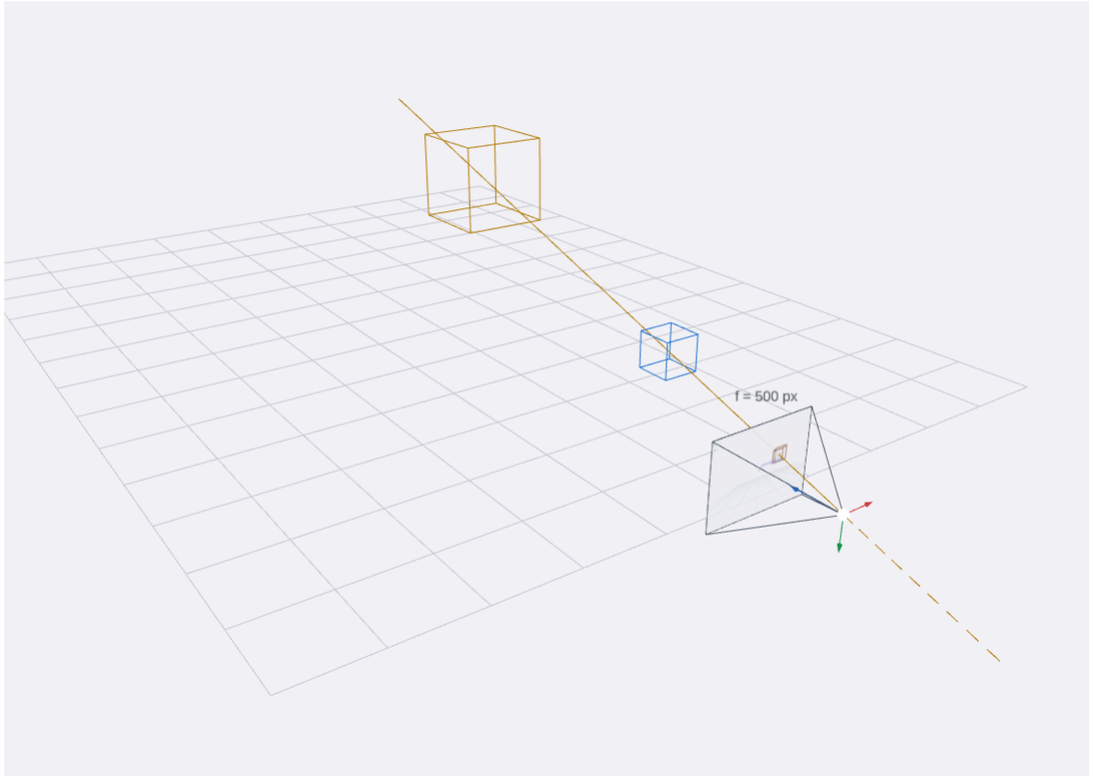
$$x_c = \frac{z_c}{f_x}(u - o_x)$$

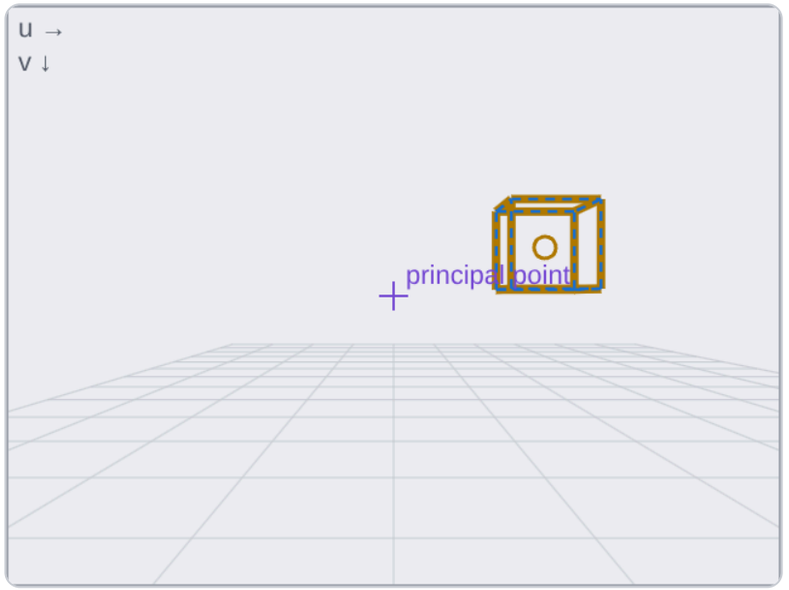
$$y_c = \frac{z_c}{f_y}(v - o_y)$$

$$z_c > 0$$

Depth is lost in projection. To get it back: a second camera (stereo, lecture 2), a depth sensor, motion, or prior knowledge of object size.

Depth is ambiguous





$$K = \begin{bmatrix} 500 & 0 & 320 \\ 0 & 500 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

pixel $(445, 200)$ → ray $z\,(0.25,\ -0.08,\ 1),\ z > 0$

A small cube close by and a cube 3× larger, 3× farther away project to **exactly the same pixels** (in the image: blue dashed on top of orange). Film-makers use this (forced perspective, the Beuchet chair); robots have to work around it.

The Beuchet chair



through the peephole: one chair



from the side: legs near, seat and back far

The peephole is a **pinhole**: from that one viewpoint, the near legs and the far seat lie on the same rays and line up into a chair.

A camera measures only **directions**. Change distances along the rays and the image does not change.

Move the eye a little (a second viewpoint) and the illusion breaks: that is the idea behind stereo, lecture 2.

Part 5

Vanishing points

Where do parallel lines meet?

A 3D line through $P_0 = (x_0, y_0, z_0)$ with direction $l = (l_x, l_y, l_z)$, in camera coordinates:

$$P(t) = P_0 + t\,l.$$

$$u(t) = f_x \frac{x_0 + t\,l_x}{z_0 + t\,l_z} + o_x \xrightarrow{t\to\infty} f_x \frac{l_x}{l_z} + o_x$$

The limit does not depend on P_0: all lines with direction l meet at the **vanishing point**

$$\tilde{u}_{vp} = K\,l$$

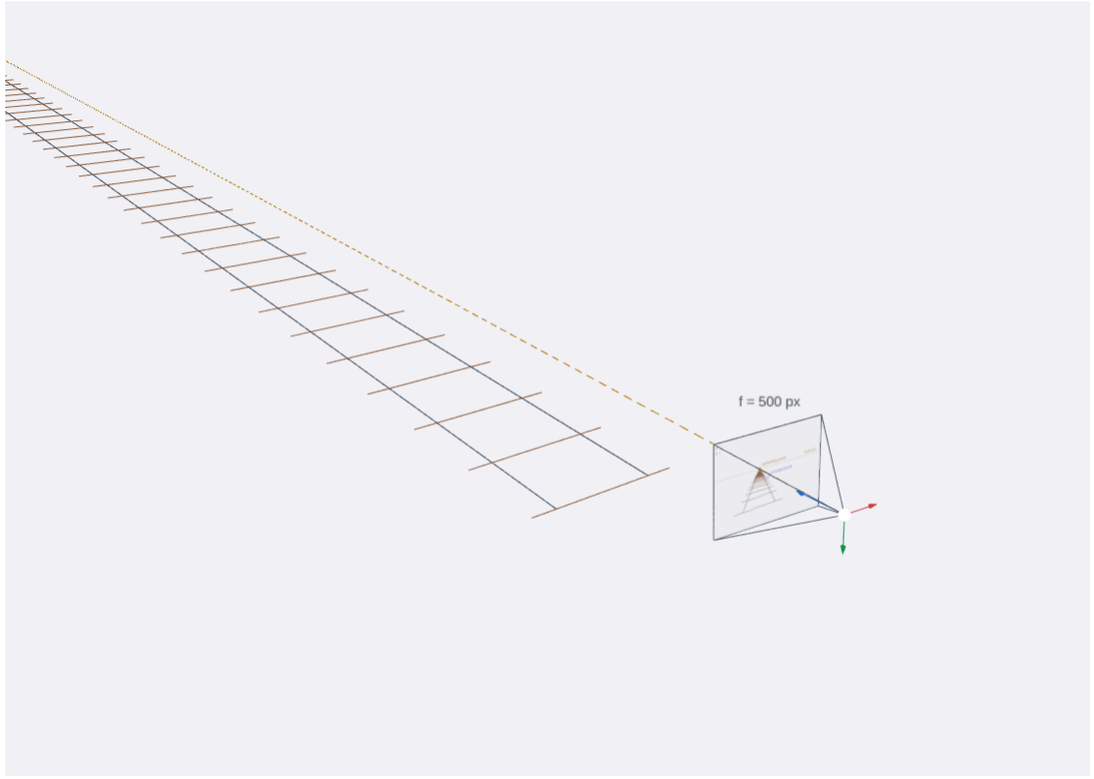
In homogeneous coordinates

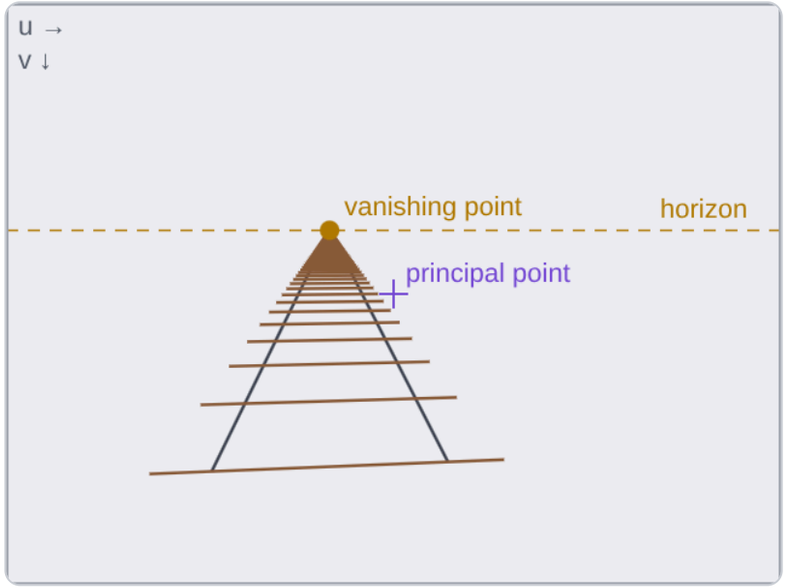
The direction l is the point at infinity $(l_x, l_y, l_z, 0)$. Projecting it: $[K \mid 0]\,(l, 0)^\top = Kl$.

If $l_z = 0$ (lines parallel to the image plane) the vanishing point is at infinity: the lines stay parallel in the image.

The vanishing point is where the ray through the pinhole **parallel to the lines** hits the image plane.

Vanishing point and horizon





$$K = \begin{bmatrix} 500 & 0 & 320 \\ 0 & 500 & 240 \\ 0 & 0 & 1 \end{bmatrix}$$

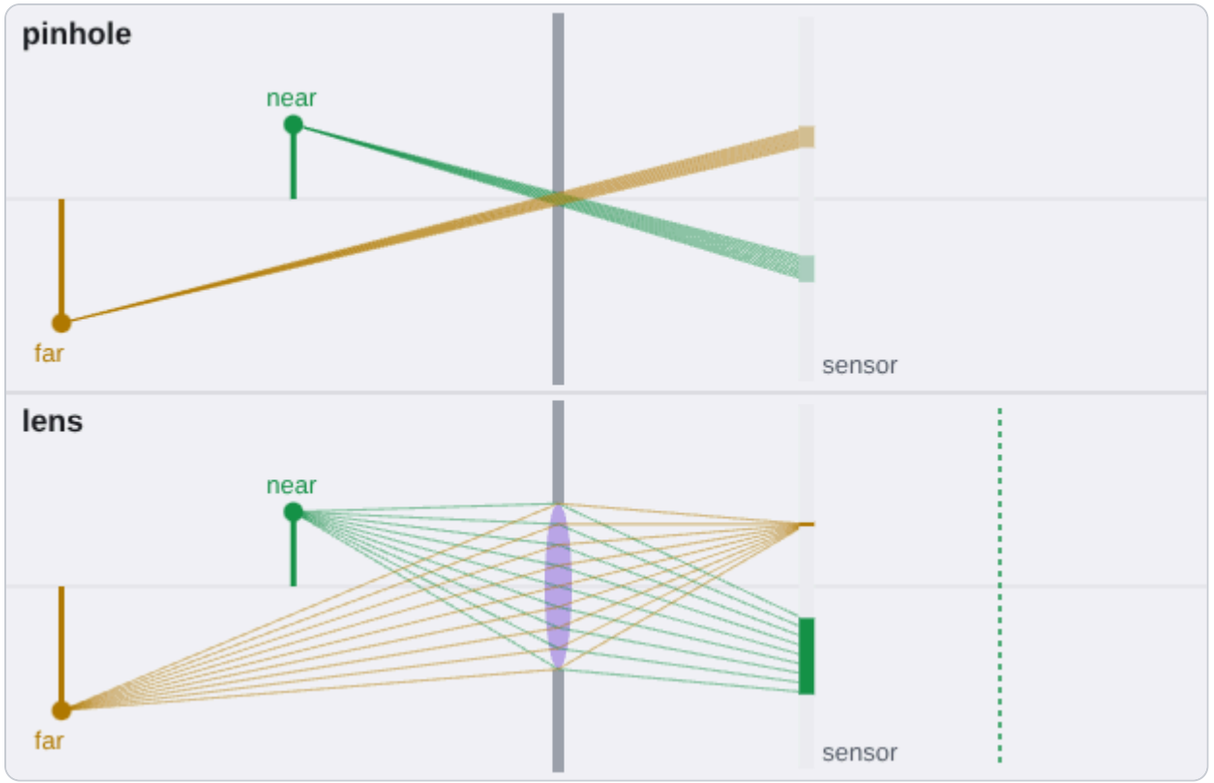
What vanishing points are good for

- **Horizon**: vanishing points of all directions in a plane lie on one line, the vanishing line; for the ground plane with normal n: $\ell = K^{-\top} n$.
- **Camera orientation**: $K^{-1}\tilde{u}_{vp}$ is the 3D direction of the lines in camera coordinates, so known directions (walls, corridors, road markings) give the camera rotation.
- **Calibration**: vanishing points of two orthogonal directions satisfy $(K^{-1}\tilde{u}_1)^\top (K^{-1}\tilde{u}_2) = 0$, a constraint on f.
- **Translation does not matter**: moving the camera does not move vanishing points; only rotation does.

Part 6

Lens distortion

Pinhole vs lens



pinhole (light ×1): near ✓ sharp · far ✓ sharp

lens (light ×14): near ✗ blur 0.44 · far ✓ sharp

dashed lines: where each object is in focus, $\frac{1}{f} = \frac{1}{a} + \frac{1}{b}$ — the nearer object focuses farther back

A pinhole keeps **every depth** sharp but lets in little light. A lens gathers far more light, but brings each depth into focus at a **different distance** behind it ($\frac{1}{f} = \frac{1}{a} + \frac{1}{b}$): focusing = moving the sensor (or the lens) so the object you care about is sharp. For an in-focus point, the lens camera behaves geometrically like a pinhole at the lens centre.

Modelling lens distortion

We keep the pinhole model and insert **one extra step**: the lens is modelled as a 2D warp of the normalised image plane, applied **between** (x, y) and the pixel (u, v).

3D point (x_c, y_c, z_c) → divide by z_c (ideal pinhole) → normalised $(x, y) = \left(\frac{x_c}{z_c}, \frac{y_c}{z_c}\right)$ → **distortion** (the lens) → distorted (x_d, y_d) → intrinsics K → pixel (u, v)

Brown–Conrady model (used by OpenCV), with $r^2 = x^2 + y^2$:

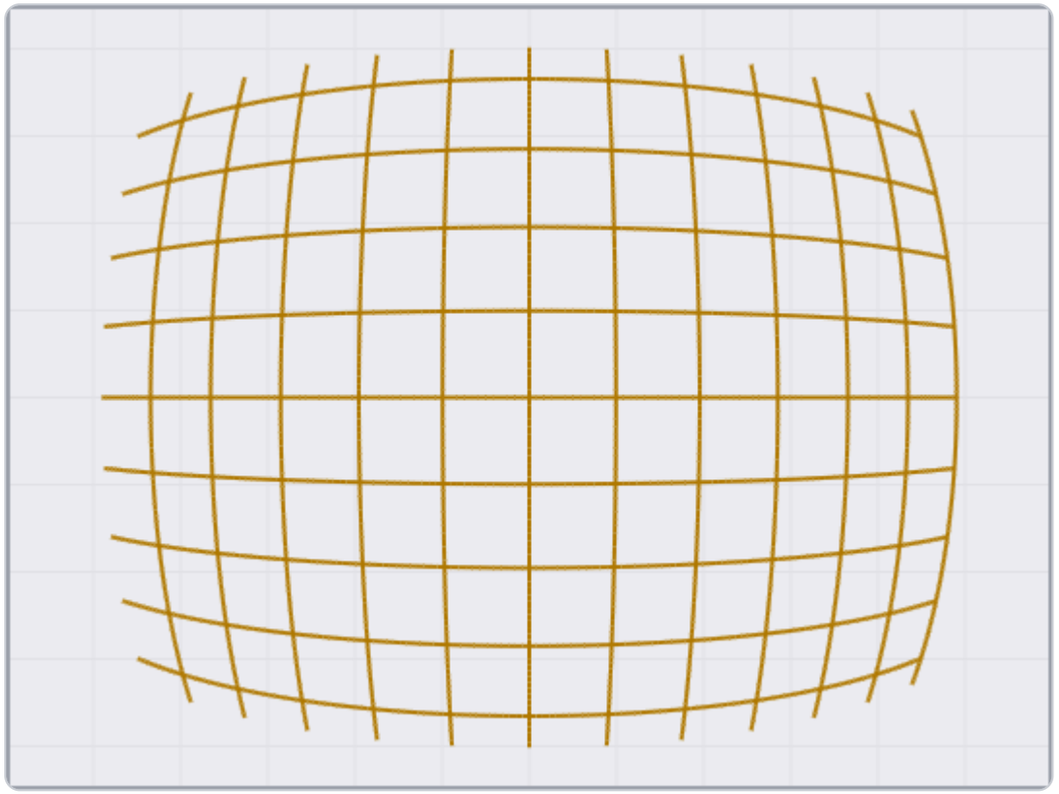
$$x_d = x\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2)$$

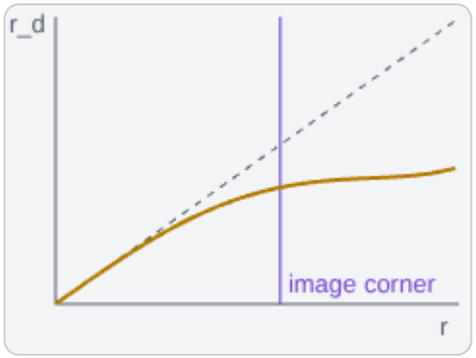
$$y_d = y\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy$$

$$u = f_x x_d + o_x, \qquad v = f_y y_d + o_y$$

- **radial** (k_1, k_2, k_3): grows with distance from the centre — barrel (wide-angle) or pincushion
- **tangential** (p_1, p_2): lens not parallel to the sensor
- no distortion: $(x_d, y_d) = (x, y)$ — back to the pinhole camera
- coefficients are estimated with K during calibration (lecture 2)
- fisheye lenses: a different model (`cv2.fisheye`)

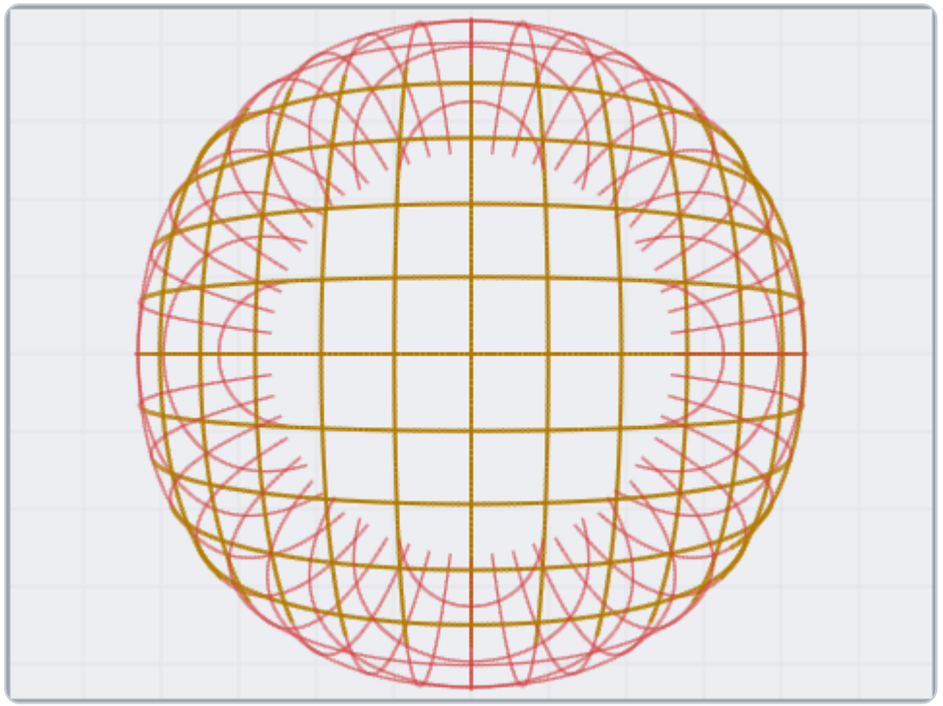
Distortion to play with

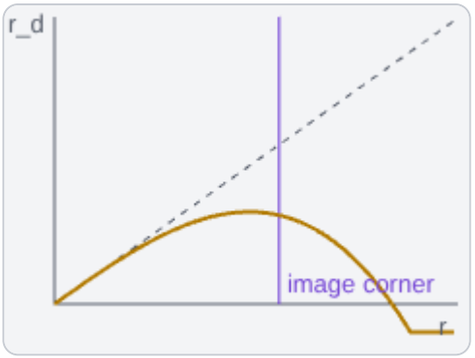




k1=-0.25 k2=0.03 k3=0
p1=0 p2=0

Undistortion and a surprising side effect





`k1=-0.35 k2=0 k3=0 p1=0 p2=0`

Undistortion inverts the polynomial — no closed form, solved iteratively (`cv2.undistortPoints`).

The polynomial is fitted only inside the image. Outside, $r_d(r)$ can turn back: points **outside** the field of view (red) land **inside** the image.

Seen in practice: opencv#17768 — objects far outside the view reappear after projection.

Rule: only project points that are within the calibrated field of view.

Summary

Transformations

- geometric image ops act on the domain: $g = f \circ T^{-1}$
- homogeneous coordinates make translation (and projection) linear
- hierarchy: rigid $\subset$ similarity $\subset$ affine $\subset$ projective
- rigid motion $x \mapsto Rx + t$, $R \in SO(3)$; 4×4 form $SE(3)$
- rotations: matrix, Euler (gimbal lock), axis–angle / rotation vector, quaternion

Camera

- $z_c\,(u, v, 1)^\top = K\,(x_c, y_c, z_c)^\top$
- pixel $\rightarrow$ ray $K^{-1}(u, v, 1)^\top$; depth is lost
- vanishing point of direction l: Kl
- lens distortion: polynomial in r; trust it only inside the image

> Practise these by hand — they come back in the labs and in the exam.

Exercises

1. Compute $R_z(90^\circ)R_x(90^\circ)$ and $R_x(90^\circ)R_z(90^\circ)$. Where does each send e_1, e_2, e_3?
2. For $R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ find the axis and angle, the rotation vector, and the quaternion.
3. Show that $T^{-1} = \begin{bmatrix} R^\top & -R^\top t \\ 0^\top & 1 \end{bmatrix}$ for $T = \begin{bmatrix} R & t \\ 0^\top & 1 \end{bmatrix}$.
4. $f_x = f_y = 600$, $(o_x, o_y) = (320, 240)$. Project $(1, 0.5, 5)$. Which 3D points project to pixel $(320, 240)$?
5. Same camera: find the vanishing point of the direction $(1, 0, 1)$. Is it inside a 640×480 image?
6. Why does translating the camera not move vanishing points?

Feedback



Please spend a minute: your feedback helps me improve the next lecture.

`tinyurl.com/mimuw-rc-feedback`

Anonymous, no login, three questions.